Explainability in Reinforcement Learning

Maxime Alaarabiou

Thursday 13th August, 2026

Chapter 1

Deep Learning

This chapter introduces the fundamental concepts of Deep Learning (DL), the training paradigm used to optimize deep Neural Network (NN). The objective is to provide the necessary foundation in DL for the ideas developed throughout the rest of this manuscript. Its structure is as follows:

Section 1.1: An overview of the historical developments that led to the emergence of the field.

Section 1.2: A formalization of the objective underlying DL algorithms.

Section 1.3: An examination of the structure of artificial NN.

Section 1.4: An explanation of how the learning process shapes NN.

Section 1.5: A discussion of how such models are evaluated.

1.1 Historical Developments

DL and NN emerged from an ambition to study the expressive capabilities of organic brain. They can be seen as a particularly striking example of biomimicry, as they draw inspiration from living systems to address scientific challenges. The first major milestone came in 1943 with the proposal of a mathematical model of an artificial neuron, with a formulation of networked interactions [McCulloch and Pitts, 1943]. This work established the key idea that networks of simple units can give rise to complex computations. The focus was on representational power, not on the ability of such systems to learn. As a result, this work remained largely theoretical and had limited practical applications.

A major step toward operationalizing the idea of NNs was achieved with the perceptron algorithm in 1958 [Rosenblatt, 1958]. It was designed to adjust its parameters from input–output pairs in order to approximate a target function. However, its fundamental limitations were exposed in 1969, showing that it could not learn functions as simple as the exclusive-or (XOR) [Minsky and Papert, 1969]. At the same time, it was demonstrated that stacking multiple perceptrons could in principle represent such functions. At the time, no effective training procedure existed for such multi-layer architectures, and it was not until 1986 that researchers proposed using gradient descent optimization to train deep NNs

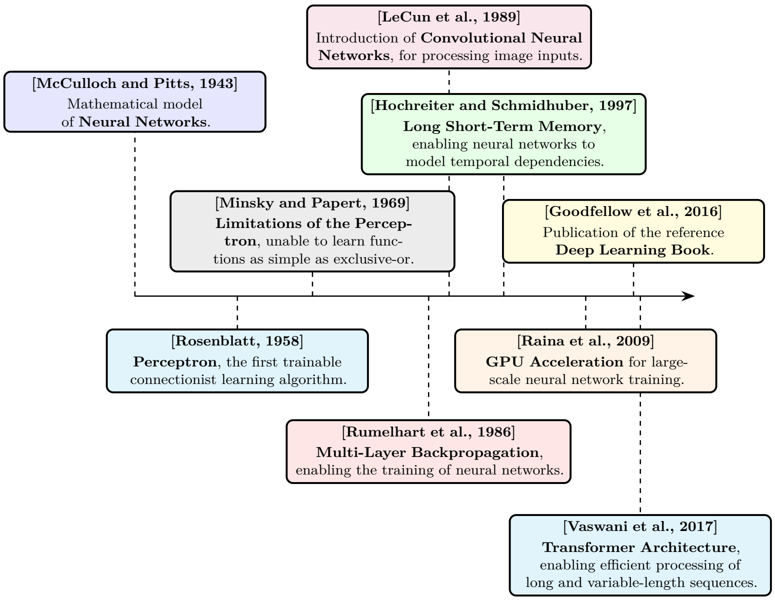


Figure 1.1: Evolution of deep learning.

[Rumelhart et al., 1986]. From this point onward, the emergence of DL is retrospectively situated, even though the term itself appeared later. DL is defined as the training of deep NNs using gradient-based optimization methods. For simplicity, the term NN will refer throughout this manuscript to deep NNs.

It took several decades before DL reached the level of prominence it has today. One major obstacle was the absence of theoretical guarantees regarding the convergence of learning algorithms. Although the universal approximation theorem shows that NN can represent a wide class of functions [Hornik et al., 1989], it did not ensure that such representations could be effectively learned through optimization methods like gradient descent. This lack of rigorous guarantees initially reduced interest within the academic community. As a result, early progress relied largely on empirical breakthroughs.

These empirical advances were made possible by two key developments: the availability of large-scale datasets and significant improvements in computational infrastructure. DL methods require vast amounts of data to perform well, and the rise of the internet alongside the digitization of information enabled the creation of such datasets [Hilbert and López, 2011]. At the same time, progress in hardware, particularly the use of graphics processing units, greatly increased computational capacity and made it feasible to train large-scale models [Krizhevsky et al., 2012].

These changes paved the way for the rise of DL as we know it today. DL has achieved remarkable success across a wide range of domains, including computer vision [Krizhevsky et al., 2012], complex games [Silver et al., 2017a], content recommendation [Covington et al., 2016], bio-chemistry [Jumper et al., 2021], as well as the now essential digital assistants based on Large Language Model (LLM) [Brown et al., 2020]. The diversity of these applications explains the enthusiasm for this technology, which continues to redefine the boundaries of what was once considered automatable.

Figure 1.1 provides a timeline of the key milestones that shaped deep learning.

1.2 Objective

Now that the main developments that led to the rise of DL have been outlined, it is necessary to formalize what is actually being achieved when performing DL. The range of problems that can be addressed with DL is extremely broad. To ensure clarity, the supervised learning setting will be considered, specifically applied to a regression task. This framework serves as a standard reference in the field, as all building blocks of DL applications are built upon this fundamental formalism.

In this setting, learning consists of progressively shaping a NN that serves as an approximation of a target function. At initialization, the network does not yet provide a meaningful representation of this function. Since a NN is a parametric function, its parameters are progressively updated during training so as to make it a better approximation of the target function. It gradually becomes capable of capturing the relationship between inputs and outputs through successive parameter updates during training. For this reason, DL is naturally situated within the framework of machine learning. A standard definition of machine learning is:

> "A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E." [Mitchell, 1997]

When this formulation is adapted to DL, the following correspondences hold:

- The computer program corresponds to the learning algorithm, which is responsible for adjusting the network's parameters.
- The task T consists of learning a NN that provides a good approximation of the target function.
- The performance measure P corresponds to a function used to evaluate the quality of this approximation.
- The experience E takes the form of a dataset composed of input–output pairs derived from the function to be approximated.

The mathematical formalism used in this chapter is now introduced in order to remove any ambiguity in the developments that follow. The function to be approximated is denoted by f. As with any function, it defines a mapping from an input to an output: $x \in \mathcal{X}$ denotes an input and $y \in \mathcal{Y}$ the corresponding output. The function f is therefore characterized by an input space $\mathcal{X}$ and an output space $\mathcal{Y}$, and can be formally written as $f : \mathcal{X} \to \mathcal{Y}$. For simplicity of exposition and notation, both $\mathcal{X}$ and $\mathcal{Y}$ are assumed to be vector spaces throughout, while keeping in mind that this framework can be readily generalized to more settings. Throughout this manuscript, the notation $\hat{\cdot}$ indicates that a given object is an approximation of another. Since the goal of the trained NN is to approximate f, this approximation is denoted by $\hat{f} : \mathcal{X} \to \mathcal{Y}$. Given an input $x \in \mathcal{X}$, the output of the approximating function $\hat{f}(x)$ is denoted by $\hat{y}$.

It is important to emphasize that the target function f is generally not known explicitly. If it were, there would be no need to learn an approximation, since the true function would already provide the exact mapping. In practice, only a finite set of observations generated by f is available, organized in what is called a dataset. To distinguish between the different samples, an index i is introduced, allowing input–output pairs of the form $(x^{(i)}, y^{(i)})$ to be defined. This leads to the following formulation for a dataset $\mathcal{D}$ of size N, where N denotes the number of available examples:

$$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}.$$

Now that the way the target function f is represented has been made explicit, one can ask what it means for a function $\hat{f}$ to be a good approximation of this function. It means that, for a given input $x^{(i)}$, the NN output $\hat{y}^{(i)}$ is close to the true output $y^{(i)}$. To quantify the quality of this approximation, a function $\mathcal{L}$ is introduced that measures the discrepancy between predictions and target values. In DL, this function called the loss function plays a central role as it guides the learning process. In regression settings, a common choice is the Mean Squared Error (MSE), defined as:

$$\mathcal{L}(\hat{y}, y) = (\hat{y} - y)^2. \tag{1.1}$$

Thanks to this loss, it is now possible to rigorously define the performance measure P. This performance measure corresponds to the average loss over the dataset and can therefore be written as:

$$P = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(\hat{f}(x^{(i)}), y^{(i)}).$$

This formulation captures the mathematical core of the learning objective. The goal is to find a NN $\hat{f}$ that minimizes the prediction error over the dataset $\mathcal{D}$. As can be seen, the setting considered here is very general, where the target function f may take many different forms. Consequently, the network $\hat{f}$ must be expressive enough to approximate a wide variety of such functions. This requirement motivates the choice of NN architectures, which are specifically designed to represent highly diverse functional mappings.

1.3 Neural Network Structure

NN are called connectionist models. Rather than modeling information processing through a set of explicit rules, they rely on a flow of signals distributed within a computational structure. Their architecture is composed of a large number of simple computational units, which operate on partial representations and exchange signals across the network. A complex pattern of weighted connections between these units enables the transformation and propagation of information.

To simplify the presentation throughout this chapter, the focus is placed on the MultiLayer Perceptron (MLP) architecture. Although more specialized architectures exist, the MLP forms a fundamental building block of modern DL. Indeed, most of the essential mechanisms of DL are present in it, making it a particularly suitable framework for introducing key concepts.

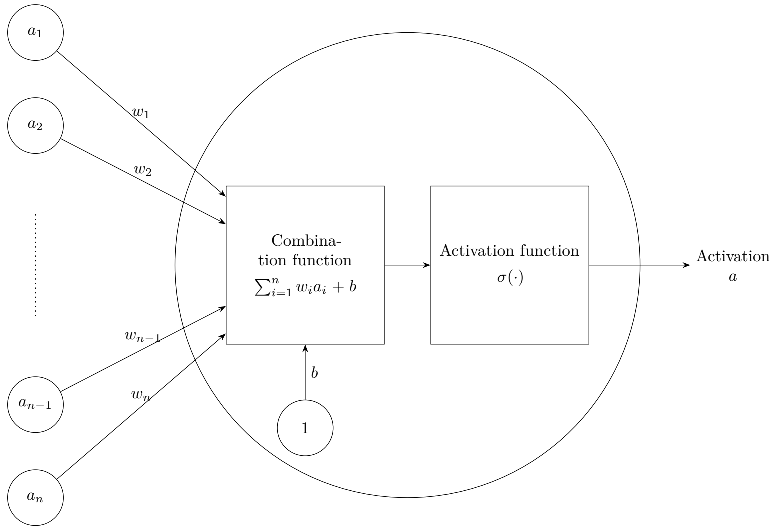


Figure 1.2: Artificiale neurone.

1.3.1 Neurons

The artificial neuron is the basic building block of NN. It is through these units that all computations within the network are carried out. A neuron receives a set of input signals from other neurons, where these inputs are represented as real-valued numbers. It produces an output, also a real-valued number, which is transmitted to subsequent neural units. Its internal state, called the activation, reflects its level of response and encodes the information processed for a given input. This activation, or lack thereof, is then used as input information by other neurons to determine their own activations, thereby propagating information through the network.

A neuron is characterized by:

- a set of incoming connections from other neurons,
- a weight associated with each of these connections,
- a bias term,
- a differentiable nonlinear activation function,
- a scalar activation state.

The activation of a neuron is determined by a weighted combination of its input signals, to which a bias is added, followed by the application of a nonlinear activation function. Mathematically, for a neuron receiving inputs $(a_1, \ldots, a_n)$, its activation a is given by:

$$a = \sigma\left(\sum_{i=1}^{n} w_i a_i + b\right),$$

where:

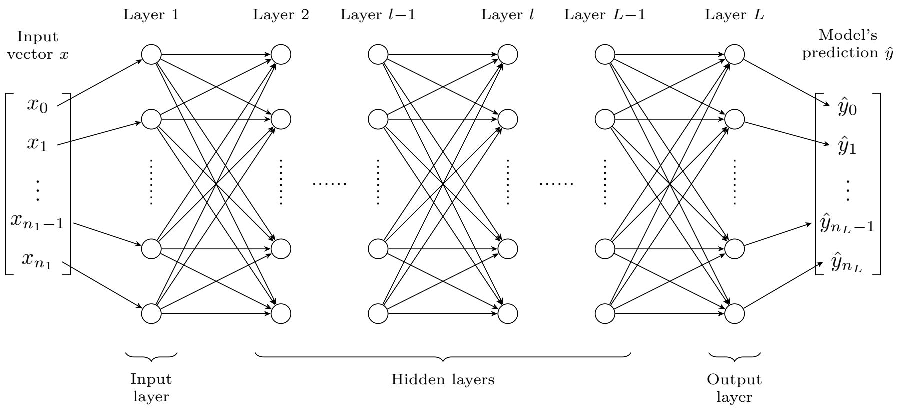


Figure 1.3: Multilayer Perceptron Architecture.

- w_i is the weight associated with the i-th incoming connection,
- b is the bias,
- σ is the activation function.

Figure 1.2 shows a schematic view of this basic unit. The set of weights $w_1, \ldots, w_i$ with the bias b constitute the parameters of the neuron, and they are the elements that evolve during training. The magnitude of a weight w (relative to the others) indicates how strongly a neuron depends on the activation of the neurons to which it is connected. A large positive weight means that a high activation in the connected neuron tends to increase the activation of the receiving neuron, whereas a negative weight implies the opposite effect. In this way, the weights modulate the strength and nature of the connections between neurons, thereby defining the overall pattern of interaction within the network.

The activation function σ models how a neuron transforms incoming signals into its output activation. Originally, this function was inspired by biological neurons, with the goal of mimicking the firing behavior of organic cells. However, modern DL no longer relies on this biological interpretation. In practice, any function that is nonlinear, differentiable, and computationally efficient can be used. Nonlinearity is a crucial property, since without it a NN would reduce to a composition of linear transformations, and would therefore be unable to approximate complex functions.

1.3.2 Layers

In MLP architectures, neurons are organized into successive layers, as illustrated in Figure 1.3. This layered structure determines how neurons are connected. In this setting, each neuron receives inputs from all neurons in the preceding layer and sends its output to all neurons in the following layer. Consequently, neurons within the same layer do not interact with each other.

Three types of layers are distinguished:

- an input layer, which directly receives input,

- one or more hidden layers (also referred to as intermediate layers), which perform intermediate transformations,
- an output layer, which produces the final prediction.

The input and output layers play a specific role. The input layer has no preceding layer: its neurons do not compute activations but instead directly encode the components of the input vector x. Conversely, the output layer has no subsequent layer, and its neurons produce the model's prediction $\hat{y}$. This structure imposes a direct correspondence between the dimensionality of the input space and the number of neurons in the input layer, and similarly between the output space and the number of neurons in the output layer.

When describing NN, especially in MLP, it is more common to reason at the level of layers rather than individual neurons. This perspective allows interactions between layers to be modeled in a matrix form. Such a representation enables efficient parallel computation and forms the basis of modern DL implementations [Krizhevsky et al., 2012]. Within this framework, it is useful to introduce the notion of layer activations. The activation of a layer corresponds to the activations of all its neurons, organized into a vector. For a layer l composed of n_l neurons, the activations are given by:

$$\mathbf{h}^{(l)} = [a_1^{(l)}, \dots, a_{n_l}^{(l)}].$$

Using this formulation, the interaction between two consecutive layers can be written compactly as:

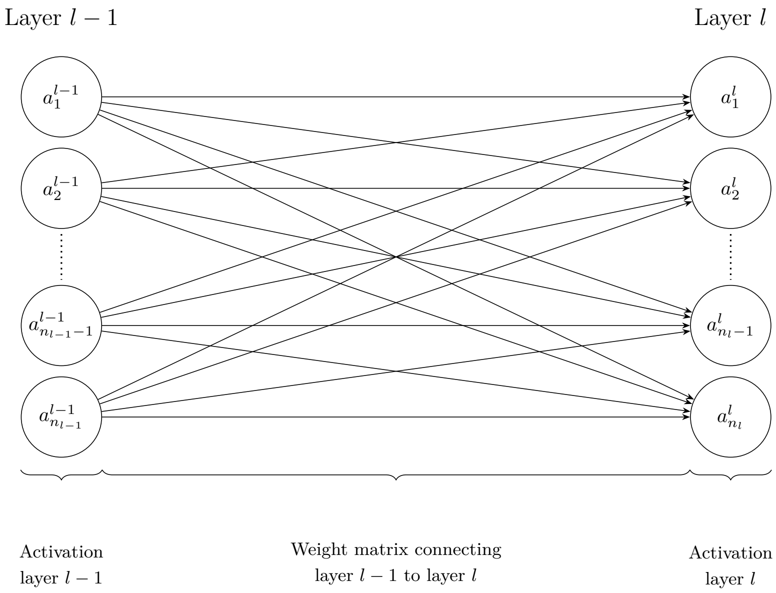
$$\mathbf{h}^{(l)} = \sigma^{(l)} \left(\mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)} \right),$$

where:

- $\mathbf{h}^{(l-1)}$ represents the activations vector of the previous layer,
- $\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ is the weight matrix connecting layer $l-1$ to layer l,
- $\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$ is the bias vector associated with layer l,
- $\sigma^{(l)}$ is the activation function applied element-wise to the neurons of layer l.

This matrix-vector representation is visualised in Figure 1.4, which highlights how each neuron in layer l receives weighted signals from all neurons in the previous layer.

It is important to examine the role played by the intermediate layers of a NN. At least one hidden layer is required to represent sufficiently complex functions. In practice, however, modern architectures often include many more. Theoretical results have shown that, for a fixed number of parameters, increasing the number of layers enhances the expressive capacity of NNs [Telgarsky, 2016]. Each layer is seen as a processing step, and more complex target functions typically require more such steps to be well approximated. However, this interpretation is based on experimental findings rather than a theoretical justification. An entire chapter of this manuscript is devoted to studying the information that can be extracted from the activations of these intermediate layers (Chapter 2).



$$h^{l-1} = \begin{bmatrix} a_1^{l-1} \\ a_2^{l-1} \\ \vdots \\ a_{n_{l-1}-1}^{l-1} \\ a_{n_{l-1}}^{l-1} \end{bmatrix} \quad W^l = \begin{bmatrix} w_{1\to 1} & w_{2\to 1} & \cdots & w_{n_{l-1}-1\to 1} & w_{n_{l-1}\to 1} \\ w_{1\to 2} & w_{2\to 2} & \cdots & w_{n_{l-1}-1\to 2} & w_{n_{l-1}\to 2} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ w_{1\to n_l-1} & w_{2\to n_l-1} & \cdots & w_{n_{l-1}-1\to n_l-1} & w_{n_{l-1}\to n_l-1} \\ w_{1\to n_l} & w_{2\to n_l} & \cdots & w_{n_{l-1}-1\to n_l} & w_{n_{l-1}\to n_l} \end{bmatrix} \quad h^l = \begin{bmatrix} a_1^{l} \\ a_2^{l} \\ \vdots \\ a_{n_l-1}^{l} \\ a_{n_l}^{l} \end{bmatrix}$$

Figure 1.4: Interaction between layers.

1.3.3 Network

With a layer-wise representation, the propagation of information can be described as a sequence of transformations applied from the input layer (indexed 1) to the output layer (indexed L):

$$\hat{f}(x) = \mathbf{h}^{(L)} = \sigma^{(L)}\left(\mathbf{W}^{(L)}\sigma^{(L-1)}\left(\dots\sigma^{(1)}\left(\mathbf{W}^{(1)}x + \mathbf{b}^{(1)}\right)\dots\right) + \mathbf{b}^{(L)}\right) = \hat{y}.$$

For a more compact description of information propagation, the layer activations are defined recursively as:

$$\mathbf{h}^{(l)} = \begin{cases} x & \text{if } l = 1, \\ \sigma^{(l)}\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right) & \text{if } 2 \leq l \leq L. \end{cases}$$

To simplify notation, it is convenient to group all parameters of the network into a single set. The model parameters are thus denoted by:

$$\theta = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{L}.$$

In this manuscript, θ is considered as a vector in $\mathbb{R}^P$, where P denotes the total number of parameters, such that $\theta = [w_1, \dots, w_P]$. The notation $\hat{f}_\theta$ expresses the dependence of the NN on its parameters. Increasing the number of parameters, either by adding more neurons per layer or by increasing the

Algorithm 1: Stochastic Gradient Descent

Input:
Dataset $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$
Architecture $\hat{f}$
Loss function $\mathcal{L}$
Learning rate $\eta > 0$
Initial parameters θ (optional)
Output:
Trained parameters θ

---

if θ *is given* **then**
    $\theta^{(1)} \leftarrow \theta$;
else
    Initialize $\theta^{(1)}$ randomly;
$n \leftarrow 1$;
while *stopping criterion not satisfied* **do**
    Select a random example $(x^{(i)}, y^{(i)}) \in \mathcal{D}$;
    Model prediction for the selected example
    $\hat{y}^{(i)} \leftarrow \hat{f}_{\theta^{(n)}}(x^{(i)})$;
    Compute the error between the prediction and the ground truth
    $\ell \leftarrow \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$;
    Compute the gradient of the error since the loss function is differentiable
    $g \leftarrow \nabla_{\theta^{(n)}} \ell$;
    Error minimization via a gradient descent step
    $\theta^{(n+1)} \leftarrow \theta^{(n)} - \eta\, g$;
    $n \leftarrow n + 1$;
Return $\theta^{(n)}$

number of layers, can enhances the expressive power of the model, allowing it to approximate more complex functions. However, this also increases computational cost and training time.

It is important to distinguish between the architecture and the parameters of a NN. When referring to the architecture of $\hat{f}$, the number of layers, the number of neurons per layer, and the choice of activation functions are considered. The parameters correspond to θ, which controls the strength of the connections between neurons.

1.4 Learning

Now that the structure of a NN has been detailed, attention must be turned to how it is modified in order to accomplish the assigned task. To do so, attention must be focused on what lies at the very core of DL: the learning process itself. As a reminder, the objective of a NN is to adjust its parameters so that it becomes a good approximation of a target function. That is, for a given architecture $\hat{f}$,

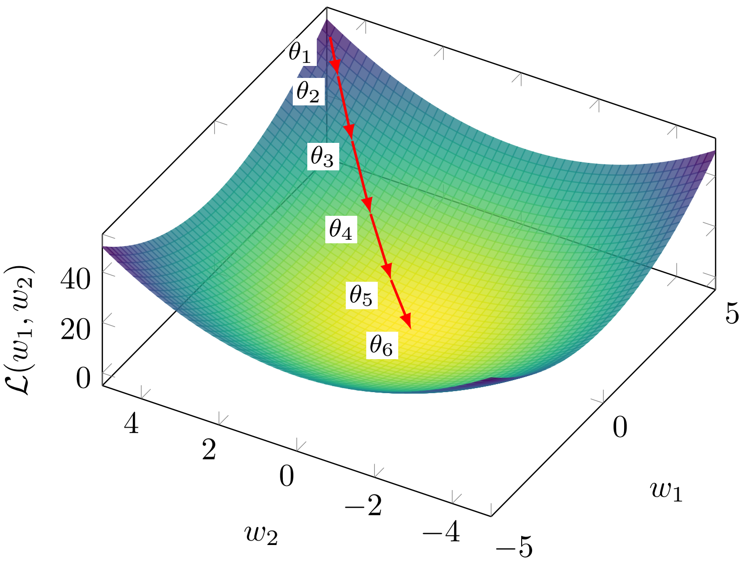


Figure 1.5: Gradient descent in a case $\mathcal{L}(w_1, w_2) = w_1^2 + w_2^2$.

the goal is to find the set of parameters θ that best approximates the target function f. Mathematically, this objective is expressed as:

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)}).$$

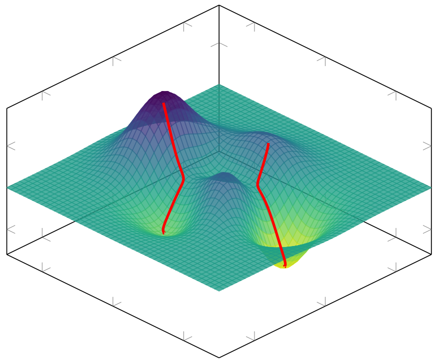
However, the NN $\hat{f}$ used to approximate the target function f is highly complex. It involves a large number of parameters as well as many nonlinear components. As a result, an exact solution cannot be found by directly studying the properties of the function. So, one must therefore rely on approximate methods to optimize NNs.

The class of algorithms used for this optimization is known as gradient descent methods. To develop an intuition, Figure 1.5 illustrates gradient descent applied to the simple convex loss function $\mathcal{L}(w_1, w_2) = w_1^2 + w_2^2$, where the red path shows the parameter updates converging to the minimum.

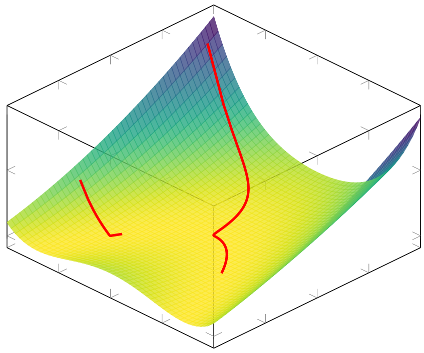
For simplicity, the focus here is on Stochastic Gradient Descent (SGD) with a batch size of 1. As in previous cases, this choice is made because this setting captures the essential intuition behind the training method.

The first step of SGD is to randomly initialize the parameter vector $\theta \in \mathbb{R}^P$ for a given network architecture $\hat{f}$. We denote this initial state by $\theta^{(1)}$. The goal is then to iteratively modify this vector in order to improve the network's ability to approximate the target function.

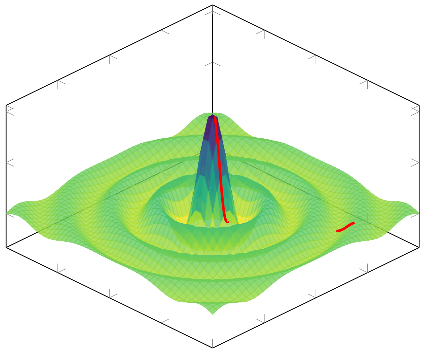
To determine how to update the parameters, the effect of small variations in each parameter on the loss $\mathcal{L}$ is studied. To analyze how parameter variations affect the loss, a fundamental mathematical tool known as the gradient is used. It is defined as the collection of partial derivatives of a function with respect to its variables. The gradient measures the effect of a small variation of each parameter on the loss.



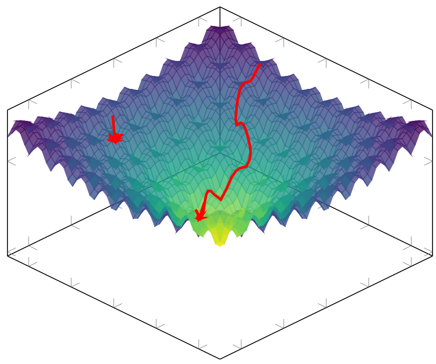
(a) Goldstein-Price



(b) Rosenbrock



(c) Sombrero



(d) Ackley

Figure 1.6: Comparison of loss surfaces and gradient descent trajectories.

In our setting, the objective is to understand, for a given example $(x^{(i)}, y^{(i)})$, how the parameters should be adjusted so that the NN prediction $\hat{y}^{(i)}$ becomes closer to the true output $y^{(i)}$. To achieve this, the partial derivatives of the loss function $\mathcal{L}$ with respect to each parameter vector θ are computed:

$$\nabla_\theta \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)}) = \left[\frac{\partial \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)})}{\partial w_1}, \ldots, \frac{\partial \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)})}{\partial w_P} \right].$$

Since both the NN and the loss function are compositions of differentiable functions, these derivatives can be efficiently computed using the chain rule. Each component of the gradient $\frac{\partial \mathcal{L}(\ldots)}{\partial w_k}$ $\forall k \in [1, \ldots, P]$ indicates how a small increase in the corresponding parameter w_k affects the loss. More precisely, for a given parameter, a positive value in the corresponding gradient component indicates that increasing this parameter will locally increase the loss. Conversely, a negative value indicates that increasing it will locally decrease the loss.

Now that information is available on how changes in the parameters affect the loss, the objective is to minimize it. To do so, the parameters are updated in the direction opposite to the gradient. Indeed, the gradient is followed in the opposite direction because it indicates the direction in which the loss increases, whereas the objective is to reduce it. This leads us to the gradient descent update rule:

$$\theta^{(n+1)} = \theta^{(n)} - \eta \nabla_{\theta^{(n)}} \mathcal{L}(\hat{f}_{\theta^{(n)}}(x^{(i)}), y^{(i)}),$$

where $\eta > 0$ is the learning rate, which controls the step size. This parameter must remain small, as the gradient only provides reliable information in a local neighborhood of the current parameters. As a result, a single step is not sufficient to significantly reduce the loss, so the process is iterated many times. Producing a sequence of parameter vectors $\theta_1, \theta_2, \ldots$ that progressively improve the model. In practice, training is stopped when successive updates no longer produce a significant decrease in the loss. Algorithm 1 summarizes this whole training procedure.

It has been mathematically shown that gradient descent applied to a convex loss with respect to the parameters converges to a global optimum, independently of the initialization. However, NN training involves highly non-convex loss surfaces, so the optimization trajectory becomes strongly dependent on the initial conditions, as illustrated in Figure 1.6, where different starting points lead to distinct local minima on standard non-convex functions.

Despite the lack of theoretical guarantees of convergence to a global optimum in this setting, the simple procedure of random initialization followed by iterative gradient-based updates is sufficient in practice to train highly complex NNs. This empirical success is often attributed to the expressive power of NNs, even though a complete theoretical understanding of this phenomenon remains an open question.

1.5 Evaluation

Now that the process of training a model has been described, the question of how to evaluate it must be addressed. In the supervised learning setting, this evaluation relies on two aspects:

- its performance on the data it was trained on,
- its performance on data it was not trained on.

The first aspect measures how well the loss minimization process has succeeded. The second measures how well the model generalizes. Indeed, the dataset $\mathcal{D}$ represents a small subset of all possible inputs. To estimate how the model will perform in general, it is important to evaluate how it performs on data it has not seen during training.

In order to quantify these two aspects, the dataset is split into two subsets:

- $\mathcal{D}_{\text{train}}$ is the set on which the model minimizes its loss during the training.
- $\mathcal{D}_{\text{test}}$ is kept aside during training in order to evaluate the model's ability to generalize.

When training is completed, the first step is to verify that the model performs well on the data it was trained on. To do so, the average loss on the training set, denoted $\bar{\ell}_{\text{train}}$, is studied:

$$\bar{\ell}_{\text{train}} = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(x^{(i)}, y^{(i)}) \in \mathcal{D}_{\text{train}}} \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)})$$

If $\bar{\ell}_{\text{train}}$ is not sufficiently low, this means that the model has not been able to learn the task. It is not necessary to go further in the analysis of the model's quality. Before restarting the training process, various design choices are then adjusted. Based on what has been discussed so far in this chapter, these include modifications to the number of layers, the number of neurons per layer, activation functions, the learning rate, and the initialization strategy. However, in practice, there are many other possible variations. There is no universal rule for choosing the appropriate value for each of them. They are therefore chosen empirically, based on models that perform well on similar tasks, combined with intuition. Their selection is often costly and remains a challenging problem in DL.

Assuming now that the model achieves satisfactory performance on the training dataset $\mathcal{D}_{\text{train}}$. We must now study its ability to generalize. To do so, the corresponding average loss on $\mathcal{D}_{\text{test}}$ is computed:

$$\bar{\ell}_{\text{test}} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(x^{(i)}, y^{(i)}) \in \mathcal{D}_{\text{test}}} \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)})$$

If $\bar{\ell}_{\text{test}} \gg \bar{\ell}_{\text{train}}$, the model fails to generalize. This situation most often arises due to a phenomenon known as overfitting. In such cases, the model does not learn general patterns but instead memorizes the training examples. This typically occurs when the number of parameters is too large relative to the amount of training data. The simplest remedy is therefore to reduce the number of parameters in the model. However, it is still necessary to ensure that the model remains sufficiently expressive to perform well on the data.

If $\bar{\ell}_{\text{test}} \approx \bar{\ell}_{\text{train}}$, this indicates that the model generalizes well beyond the training data [Kawaguchi et al., 2022]. The model can therefore be used with confidence for the task on which it was trained.

The ability of a neural network to generalize provides evidence that the model has learned more than a simple memorization of the training examples. Instead, it must construct internal representations that capture regularities present in the data. These representations emerge progressively through the hidden layers during training. Understanding their structure is therefore a key step toward understanding how neural networks generalize. The collection of these representations is referred as the Latent Space (LS). The next chapter is devoted to the study of these LSs.

Chapter 2

Latent Space Analysis

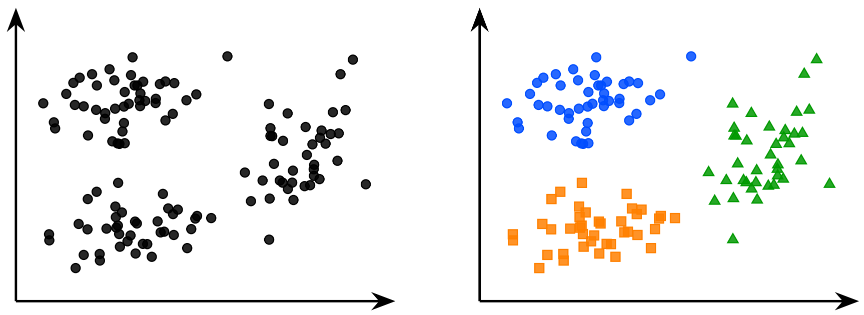
In classical machine learning approaches, practitioners had to manually design data representations in order to make them usable by simple models. Deep Learning (DL) introduces a paradigm shift: rather than relying on hand-crafted features to meaningfully represent data, it automatically learns these representations. Through the successive activation of hidden layers, Neural Network (NN) gradually construct representations that become increasingly relevant to the task under consideration [Rumelhart et al., 1986]. These representations are not meaningful at initialization, they emerge progressively during training as the model optimizes its objective. It is precisely through these learned representations that a trained network is able to map an input to an output, including for examples never encountered during training. Since these representations are learned internally by the network and are not directly observable from the input or output spaces, they are commonly referred to as Latent Representation (LR). The collection of these LR learned throughout the network is commonly referred to as a Latent Space (LS).

These LS are now at the core of a growing number of practical applications. Neural video compression exploits the compactness of LS to encode information efficiently [Lu et al., 2019]. In Reinforcement Learning (RL), agent internal representations are used for efficient exploration in extremely open environments [Burda et al., 2018]. Recommendation systems model users and content within a shared LS to compute meaningful similarities [He et al., 2017]. Retrieval-augmented generation systems index and query knowledge bases through latent embeddings [Lewis et al., 2021]. Finally, Large Language Model (LLM) and multimodal architectures rely on the quality of learned representations to align heterogeneous modalities such as text, images, and audio [Radford et al., 2021]. These examples represent only some of the most widespread applications of LR.

Given their extensive use, the study of LR has become a central topic in DL research. A major objective is to understand how information is organized within these spaces, which concepts are encoded, and how these representations influence model behavior.

- Section 2.1 introduces the working hypothesis that underlies much of the research on LS analysis, which we refer to as the concept hypothesis (Hypothesis 1).

- Section 2.2 presents probing methods for identifying and extracting con-



(a) Observed activations **(b)** Under the concept hypothesis

Figure 2.1: The same latent space activations seen without concept hypothesis **(a)** and under the working hypothesis that concepts emerge in the latent space **(b)**. Colors represent hypothetical concept memberships assigned by the model; they are not observed labels. Under this hypothesis the structure of the latent space becomes meaningful and amenable to analysis.

cepts from LR.

- Section 2.3 reviews semantic direction-based approaches.
- Section 2.4 discusses sparse autoencoders.
- Section 2.5 covers clustering-based approaches.
- Section 2.6 provides a comparative synthesis of these methods and discusses the open questions that remain.

2.1 Concept Hypothesis

In this section, we introduce the main working hypothesis underlying much of the literature on LS analysis. Although rarely stated explicitly, this hypothesis is implicitly present in the recurring use of notions such as abstraction, representation, LS, concept, and semantics. Throughout this manuscript, we refer to this hypothesis as *Concept Hypothesis* (Hyp. 1). The goal of this section is to make this conceptual framework explicit by defining its main components.

The *Concept Hypothesis* (Hyp. 1) originates from the need to explain the generalization ability of deep NN observed after training, as discussed in Section 1.5. A model is said to generalize when, for an input $x \in \mathcal{X}$ not seen during training, it still produces an output $\hat{y}$ close to the true output y. Such behavior cannot be explained by simple memorization of training samples. Instead, the model must extract and exploit underlying regularities in the target function. This requires a process of abstraction, as defined in Definition 2.1.1, whereby raw inputs are transformed into internal structures that are meaningful for the task [Bengio et al., 2014].

Definition 2.1.1: Abstraction

Abstraction is the process by which a model transforms an input into a task-relevant internal form. It captures and restructures information in order to represent relationships that are useful for prediction or decision-making.

The ability of NN to perform abstraction arises from their intermediate activations. A NN does not directly map x to y, as illustrated in Figure 1.3, it computes a sequence of intermediate activations across hidden layers. These activations are numerical encodings of the input referred to as embeddings. Once training is complete, these embeddings are structured to support the performance of a specific task. To mark this distinction, we will refer to these trained embeddings as LR, as defined in Definition 2.1.2.

Definition 2.1.2: Latent Representation

A representation is the internal encoding of an input given by a intermediate layer of a trained NN.

Each hidden layer refines the representation of the previous layer. Each transformation progressively reshapes the representation into a form more directly aligned with the output space [Rumelhart et al., 1986]. This process can be described as a sequence of transformations where each $h^{(l)}$ is a representation:

$$x \to h^{(1)} \to h^{(2)} \to \cdots \to h^{(L-1)} \to \hat{y}$$

The set of all such representations produced by a layer over a dataset defines its LS, as defined in Definition 2.1.3.

Definition 2.1.3: Latent Space

The LS is the set of all representations produced by a layer over a dataset. It is shaped by learning and organizes representations according to task-relevant properties.

Under the *Concept Hypothesis* (Hyp. 1), the analysis of the LS is assumed to provide access to abstract structures learned by the model, as illustrated in Figure 2.1. These structures are referred to as concepts. In this manuscript, a concept is defined as an abstract representation that organizes information and supports reasoning (see Definition 2.1.4).

Definition 2.1.4: Concept

A concept is a task-dependent abstraction that captures regularities in the input space relevant for predicting the target output.

Our definition of concepts is consistent with both the pragmatic tradition in philosophy and the information-processing view of cognition. Within these frameworks, concepts arise from the need to simplify an otherwise intractable reality. Because the environment contains more information than any cognitive system can process, intelligent behavior relies on abstractions, called concepts, that discard irrelevant details while preserving the information required to achieve a given objective [Peirce, 1878, Misak, 2013]. These concepts are realized as internal representations that reduce complexity and enable reasoning under limited computational resources [Newell, 1980, Simon, 1996]. Although these theories were originally developed to explain human cognition, the *Concept Hypothesis* (Hyp. 1) assumes that the same principles can be extended to deep NN.

In order to influence the model's computations, concepts must necessarily admit a concrete realization within the LS. We refer to this realization as the Concept Structure (CS). The definition of the CS is provided in Definition 2.1.5.

Definition 2.1.5: Concept Structure

A CS is the specific form for how a concept is instantiated within the LS.

The methods reviewed in this chapter all build on the *Concept Hypothesis* (Hyp. 1) formalized in Hypothesis 1, but differ in how they characterize the CS of a concept. This choice directly determines how concepts can be identified and extracted from the LS. As a result, several families of LS analysis methods have emerged in the literature, each adopting a different view of how concepts are structured within the LS.

Hypothesis 1: Concept Hypothesis

Neural networks generalize by performing abstraction. Through training, their LS becomes organized around concepts. Concepts are task-dependent abstractions that capture regularities relevant to the task. Analyzing the LS therefore provides access to the CS through which these concepts are instantiated in the model.

Now that the definition of a *Concept Hypothesis* (Hyp. 1) has been established, we can consider its relationship to human. According to our definition, we can imagine concepts that are useful for the machine but do not make sense to humans. This relationship is captured by the notion of semantics [Marconato et al., 2023], defined in Definition 2.1.6. Some concepts developed in LS can be readily associated with human concepts and are said to have high semantic value. Others may contribute to task performance while remaining difficult for humans to understand, and are said to have low semantic value.

Definition 2.1.6: Semantics

Semantics refers to the alignment between concepts in the model LS and concepts used by a human observer. A concept is semantic if it can be associated with a human interpretable concept.

It is important to note that this conceptual framework is not supported by a complete theoretical understanding of deep NN. Current mathematical tools do not provide a rigorous characterization of their internal representations and do not formally prove that concepts emerge as identifiable CSs within LS. Nevertheless, this perspective constitutes a widely adopted working hypothesis in the field of representation and LS analysis.

The remainder of this chapter is devoted to reviewing methods for analyzing LS and extracting the concepts they encode. We begin with probing, one of the

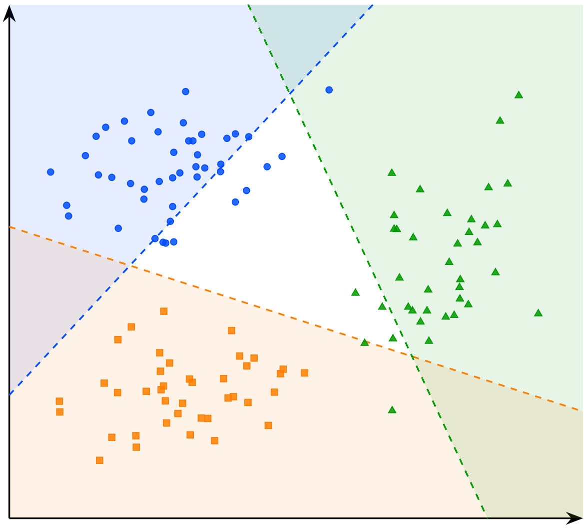


Figure 2.2: Decision boundaries of the linear probes for concepts A, B, and C. The colored regions indicate the areas classified positively by each probe.

most direct approaches for assessing whether concepts are represented in the LS.

2.2 Probing

Probing methods aim to determine whether a given semantic concept is encoded within the LS of a NN [Li et al., 2024, Karvonen, 2024]. More specifically, they investigate whether the presence or absence of a selected concept can be inferred from embeddings. The underlying assumption is that if a concept can be reliably decoded from these representations, then the network likely encodes information related to it in order to perform its task.

A probing analysis begins by identifying a set of semantic concepts that the network could potentially use to accomplish its task. Each example in the dataset is then annotated to indicate the presence or absence of the studied concepts. We denote by c_j the presence or absence of concept j in example x. The dataset can therefore be written as follows:

$$\mathcal{D} = \{(x^{(i)}, y^{(i)}, c_1^{(i)}, \dots, c_n^{(i)})\}_{i=1}^{N}.$$

The data are subsequently projected into the LS of the pretrained model. We denote by h^l the representation vector obtained at layer l. From these LR, a classifier is trained in a supervised manner to predict the presence of the considered concept. This classifier is referred to as a probe and is denoted by p. Its purpose is to determine whether a given concept is present in the representation. Its purpose is therefore to assess whether information associated with the concept is accessible from the network's internal representations. This can be formalized as follows:

$$p_\theta^{l,j} : h^l \mapsto c_j, \quad \theta^* = \arg\min_\theta \mathcal{L}\big(p_\theta^{l,j}(h^l), c_j\big)$$

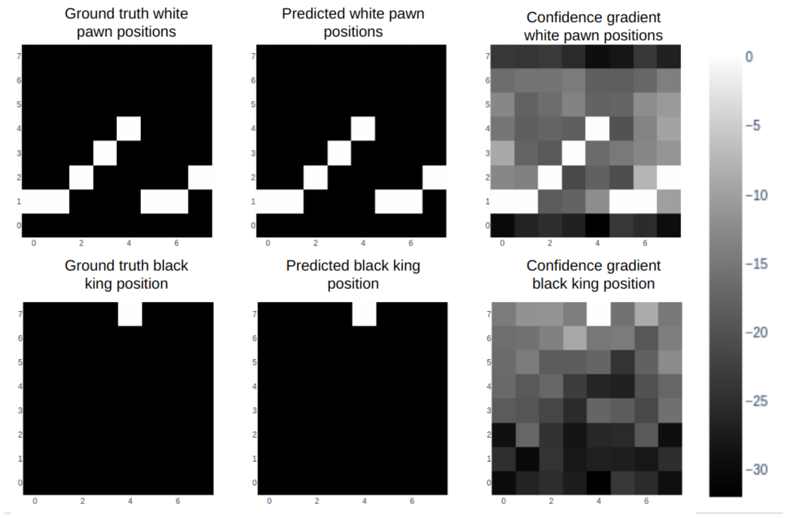


Figure 2.3: Probing analysis of a large language model trained to play chess [Karvonen, 2024]. Each row corresponds to a piece type: white pawns (top) and the black king (bottom). The left column shows the ground truth board positions, the middle column shows the positions predicted by the probe from the model's internal representations, and the right column shows the confidence gradient across board squares. The close agreement between ground truth and predicted positions suggests that the model internally maintains a structured representation of the board state, even though it was only trained on sequences of moves expressed in natural language.

The classifier used to detect the presence or absence of a concept must remain intentionally simple. Indeed, if the probe has too much capacity, it may reconstruct the target information from complex correlations that do not necessarily correspond to information explicitly encoded in the LS. In such a case, it becomes difficult to determine whether the concept is genuinely represented by the network or merely reconstructed through the expressive power of the classifier. For this reason, probing methods often rely on linear classifiers. A linear separation suggests that the information is easily accessible to the NN and potentially usable during decision-making. If the classifier achieves performance significantly above chance level, this suggests that the LS contains exploitable information related to the considered concept.

An application of this technique can be found in studies investigating whether LLM develop internal representations of their environment [Li et al., 2024, Karvonen, 2024]. In these works, researchers train LLM to play games such as chess or Othello in a purely textual setting. The model is provided only with sequences of moves and information about the actions it can take, all expressed in textual form. The central question is whether the LLM develops an internal representation of the game state. To address this question, the probed concepts correspond to the presence or absence of different game pieces on specific board positions. As illustrated in Figure 2.3, probes are able to reconstruct the full board state from the model's hidden representations, even though the only available information to the model is a sequence of text. This suggests that the model does not merely learn statistical associations from the training data, but instead develops a structured representation of the underlying environment in which the data are generated.

However, the fact that a concept can be decoded from LR does not necessarily imply that the model actually uses this information to perform its task. To assess whether representations play a causal role in the model's output, it is possible to perform interventions on the LR. An intervention consists of modifying a LR in a controlled manner by activating or deactivating a given concept, and then studying its impact on the model's output. If the resulting change in the model's output is consistent with the applied perturbation, this suggests that the representation had a causal role.

To this end, the LR h^l is minimally modified such that the probe prediction for the target concept c_j changes state, while the predictions associated with other concepts remain unchanged. This goal can be formulated as an optimization problem over the latent activation itself. The optimization is performed directly on the representation $h^{l'}$, using gradient-based optimization as introduced in Section 1.4. The solution to this problem is the optimal modified representation h^{l*}, defined as:

$$h^{l*} = \arg\min_{h^{l'}} \underbrace{\gamma\, \mathcal{L}_{\text{flip}}\big(p_\theta^{l,j}(h^{l'}), c_j\big)}_{\text{flip target concept } c_j} + \underbrace{\lambda \sum_{k \neq j} \mathcal{L}_{\text{preserve}}\big(p_\theta^{l,k}(h^{l'}), c_k\big)}_{\text{preserve other concepts } c_{k\neq j}} + \underbrace{\beta \|h^{l'} - h^l\|_2^2}_{\text{minimality}}$$

where:

- h^l is the original LR of the input at layer l,
- $h^{l'}$ is the modified LR being optimized,
- h^{l*} is the optimal modified representation,
- $\mathcal{L}_{\text{flip}}$: loss encouraging the prediction of the target concept c_j to change state (i.e., flipping the predicted label),
- $\mathcal{L}_{\text{preserve}}$: loss enforcing invariance of all non-target concepts c_k for $k \neq j$,
- $\|h^{l'} - h^l\|_2^2$: regularization term ensuring the modified representation remains close to the original LR,
- γ, λ, β: weighting coefficients controlling the trade-off between the intervention strength, the preservation of other concepts, and the minimality of the perturbation.

In the context of LLM studied previously, such interventions have been used to remove specific pieces from the model's internal representation of the game state [Li et al., 2024, Karvonen, 2024]. The main observation is that, in the vast majority of cases, the model behaves as if the piece had effectively disappeared. The LLM continues to generate legal moves while correctly accounting for the absence of the removed piece. This provides evidence for the causal role of these internal representations in the model's behavior.

A main limitation of the probing approach is that the concepts under investigation must be predefined before any analysis can be performed. This introduces a strong bias in what can be studied. Moreover, annotating a dataset

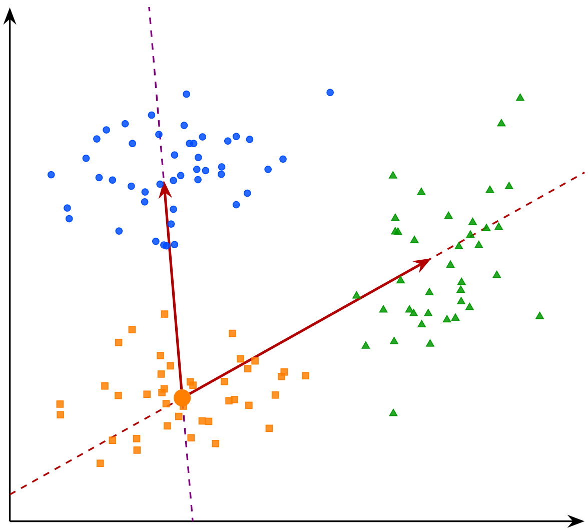


Figure 2.4: Interpretable directions in the latent space. The arrows represent semantic shifts from concept B toward concepts A and C. Moving along these directions in the latent space induces a coherent change in the model's behavior, suggesting that the transition between concepts is continuously encoded along these axes.

for each concept is extremely time-consuming and requires significant human effort. Interventions in LS are also difficult to interpret, as it is not always clear what constitutes a meaningful change in the network's output under a given perturbation.

Moreover, probing methods provide a binary view of concepts, focusing on whether a given concept is present or absent in a LR. This binary perspective can be overly restrictive, as concepts may be represented in a more continuous or graded manner, with varying degrees of presence. To obtain a more detailed characterization of these representations, other approaches have been proposed, such as the analysis of interpretable directions in LS.

2.3 Interpretable Directions

To overcome the limitations of binary concept detection in probing methods, several approaches have been proposed to analyze the structure of LR. Among them, the notion of interpretable directions has emerged as a natural extension, aiming to capture the continuous nature of how concepts are encoded in representation spaces. Empirically, it has been observed that shifting a LR along a direction can induce coherent and semantically meaningful changes in the model's output [Haas et al., 2024, Voynov and Babenko, 2020]. Such transformations can be mathematically express as:

$$h' = h + \alpha \mathbf{v}_i,$$

where:

- h denotes the original embedding,

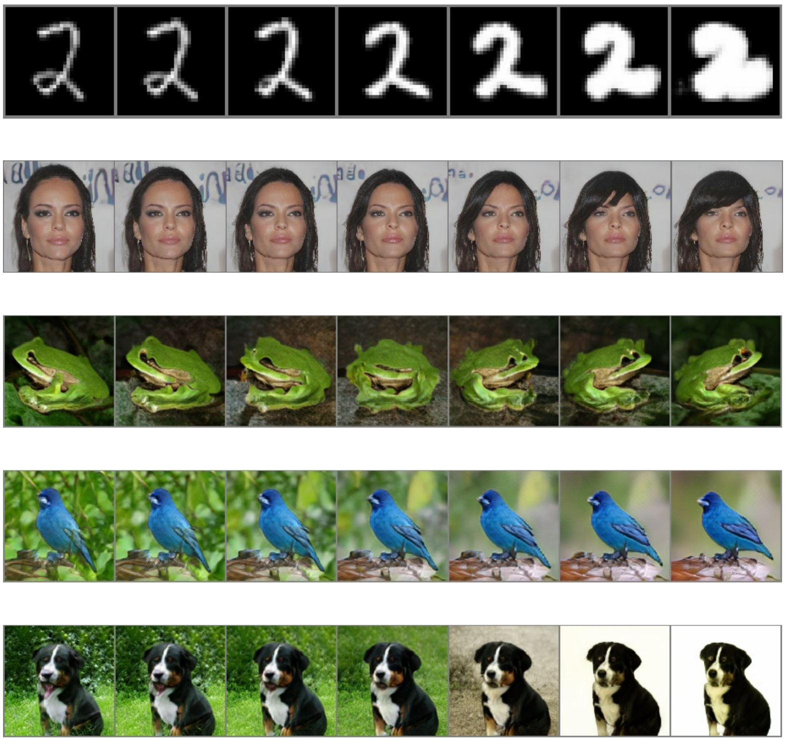


Figure 2.5: Interpretable directions in the latent space of a generative model [Voynov and Babenko, 2020]. Each row corresponds to a distinct direction $\mathbf{v}_i$, and each column to an increasing value of α. The resulting transformations are semantically coherent and diverse: stroke thickness for digits, head rotation for faces, texture and background for natural images.

- $\mathbf{v}_i$ is a unit vector associated with concept i,
- α controls the strength of the intervention,
- h' is the resulting perturbed representation.

In many cases, the resulting change in the model's output varies smoothly with α, suggesting an approximately linear CS with respect to certain semantic factors, as illustrated in Figure 2.5. This observation has led to the hypothesis that neural representations may organize concepts in a partially linear manner within the embedding space. Vectors of this form are referred to as interpretable directions, as they correspond to transformations that can be associated with specific semantic attributes. The objective of this line of work is therefore to identify directions in LS that align with meaningful variations in the encoded concepts.

This phenomenon has been extensively studied in generative image models. As illustrated in Figure 2.5, controlled perturbations of LR along interpretable directions can induce specific semantic transformations, such as stroke thickness modification in digit images, head rotation in facial images, or texture and

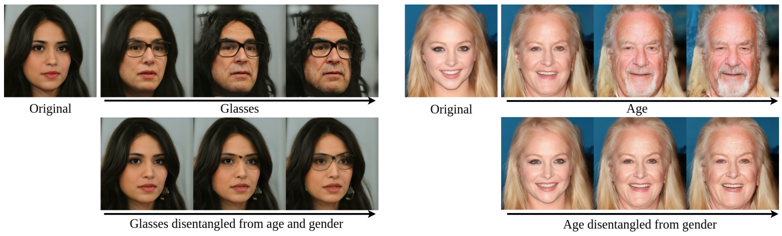


Figure 2.6: Illustration of entanglement in the latent space of a generative face model [Haas et al., 2024]. The top rows show entangled directions: moving along the glasses direction simultaneously modifies age and gender, and moving along the age direction also alters gender. This reflects the entanglement phenomenon, where a single direction in latent space induces unintended changes across multiple semantic concepts at once. The bottom rows show the corresponding disentangled directions, where each axis controls a single semantic attribute while leaving the others unchanged.

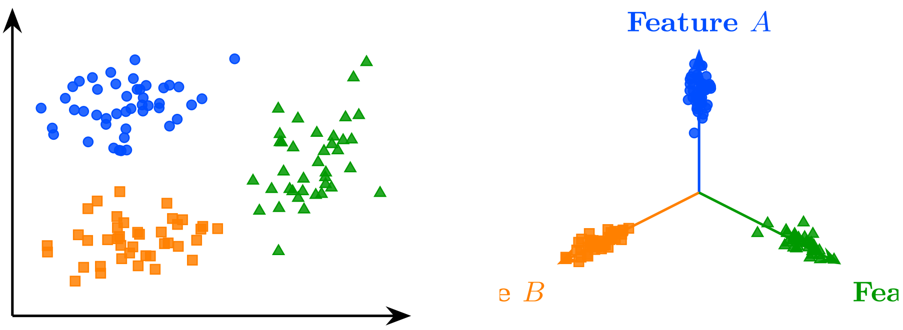
background changes in natural images [Voynov and Babenko, 2020]. These observations have led to the development of the field of LS editing, whose objective is to manipulate generated content directly through modifications in LS rather than through changes in the input prompt.

A central challenge remains identifying such interpretable directions. Most existing approaches rely on auxiliary models that encode human defined semantics, such as CLIP [Haas et al., 2024], to relate latent variations to textual concepts. Consequently, these methods inherit a limitation similar to that of probing techniques, since concept discovery depends on a pre-existing semantic framework defined by humans.

To overcome this limitation, a research direction known as unsupervised semantic discovery has emerged. Its goal is to uncover interpretable directions in LS without explicit semantic supervision. In these approaches, one model applies perturbations along random directions in LS, while a second model attempts to recognize the applied transformation. If a stable correspondence emerges between the perturbation and its prediction, this suggests that the considered direction corresponds to a coherent CS [Voynov and Babenko, 2020]. In this framework, human intervention occurs only a posteriori, to verify whether the discovered directions indeed correspond to interpretable semantic variations.

Despite these advances, several challenges remain. It appears that not all concepts encoded by neural networks are linearly identifiable in LS. In practice, interpretable direction methods typically discover only a few dozen directions per dataset, which intuitively seems insufficient to account for the full diversity of semantic variations that a generative model is capable of producing.

In addition, LR often exhibit a phenomenon known as entanglement, illustrated in Figure 2.6, where multiple semantic attributes are mixed within the same directions of the representation space. Ideally, one would like each semantic factor to vary independently along a specific direction, without affecting other attributes. In practice, this is rarely the case, as a single direction in LS often encodes multiple semantic concepts simultaneously, such that modifying one concept can unintentionally affect several others. This lack of separability indicates that semantic information is not cleanly factored in the representation



(a) Dense latent space **(b)** Sparse SAE space

Figure 2.7: Transformation from a dense latent space to a sparse SAE-style representation. On the left **(a)**, embeddings are entangled in the initial latent space. On the right **(b)**, the sparse representation aligns each concept with a dedicated feature axis.

space, which motivates the search for more structured decompositions. This limitation has motivated a stronger hypothesis about how NN organize information internally, namely the *Superposition Hypothesis* (Hyp. 2), introduced in Section 2.4. Under this hypothesis, Sparse AutoEncoderss (SAEs) provide a means of analyzing LRs.

2.4 Sparse Autoencoder

Approaches based on latent directions face limitations. Only a restricted subset of concepts admit a linear representation, and even these directions are often entangled with multiple semantic concepts, as illustrated in Figure 2.6. To address this limitation, an extension of the *Concept Hypothesis* (Hyp. 1) has been proposed under the name *Superposition Hypothesis* (Hyp. 2), formalized in Hypothesis 2 [Elhage et al., 2022].

According to this hypothesis, in an unconstrained LS, each concept is represented in its own independent linear direction. However, NN operate in finite-dimensional spaces, so when the number of concepts exceeds the available dimensions, such a one-to-one correspondence becomes impractical. The network must therefore reuse its representational dimensions, allowing multiple concepts to be encoded using the same directions. This shared encoding is referred to as superposition [Bricken et al., 2023]. As a result, in a superposed representation, linear analysis of the LS may not isolate individual concepts, the resulting directions reflect a mixture of several semantic attributes. For the *Superposition Hypothesis* (Hyp. 2), this provides an explanation for the entanglement observed when analyzing LS.

Figure 2.8: A concept direction corresponding to the Golden Gate Bridge, discovered by a sparse autoencoder applied to a large language model [Templeton et al., 2024]. This direction activates strongly whenever the Golden Gate Bridge is evoked in the input, whether through English text, other languages, or even images, demonstrating that the learned concept is not tied to a specific surface form but captures a deeper semantic abstraction shared across modalities.

Hypothesis 2: Superposition Hypothesis

Concepts are preferentially represented by distinct linear directions in LS. When the number of concepts exceeds the available dimensions, multiple concepts are encoded in superposition within the same directions.

The idealized non-superposed representation and can be mathematically expressed as follows:

$$\mathbf{h} = \sum_{i=1}^{n} \alpha_i \mathbf{v}_i, \qquad \forall i \neq j, \quad \mathbf{v}_i^\top \mathbf{v}_j \approx 0, \qquad \|\boldsymbol{\alpha}\|_0 \ll n,$$

where:

- $\mathbf{h}$ denotes a LR,
- n is the total number of concepts,
- $\mathbf{v}_i$ represents the direction associated with concept c_i,
- α_i corresponds to the activation strength of concept c_i in the representation $\mathbf{h}$,
- $\mathbf{v}_i^\top \mathbf{v}_j \approx 0$ indicates that concept directions are approximately orthogonal, meaning each concept independently controls a single factor of variation,
- $\|\boldsymbol{\alpha}\|_0 \ll n$ indicates that only a small fraction of concepts are active for a given representation.

Under the *Superposition Hypothesis* (Hyp. 2), the phenomenon of superposition becomes an obstacle for any method that attempts to analyze LR directly in the original activation space. This has motivated the development of methods that aim to recover a less superposed LR. A common approach is to map LS into a higher-dimensional space while imposing additional constraints that encourage individual concepts to be represented separately. SAE provide a practical implementation of this approach and can be defined as follows:

$$z = f_{\text{enc}}(h), \quad \hat{h} = f_{\text{dec}}(z)$$
$$\mathcal{L}_{\text{auto}} = \gamma \|h - \hat{h}\|_2^2 + \lambda \|z\|_1$$

where:

- $f_{\text{enc}}(h)$ is the encoding function that projects the original representation h into a higher-dimensional sparse space,
- $f_{\text{dec}}(z)$ is the decoding function that reconstructs the original representation from the sparse code,
- $z \in \mathcal{Z}$ is the sparse representation of h, with $\dim(z) \gg \dim(h)$,
- $\hat{h}$ is the reconstructed representation,
- $\gamma \|h - \hat{h}\|_2^2$ is the reconstruction loss ensuring that the original representation can be faithfully recovered,
- $\lambda \|z\|_1$ is the sparsity regularization term that encourages most components of z to be zero,
- γ, λ are weighting coefficients controlling the trade-off between reconstruction fidelity and sparsity.

In practice, both f_{enc} and f_{dec} are implemented as linear projections between the original representation space and the high-dimensional space $\mathcal{Z}$. This design follows the same principle as in probing methods, the transformation used to analyze the representation should remain as simple as possible. More expressive projection functions could themselves introduce complex structures into the transformed representation. In that case, it would become difficult to determine whether the observed concept reflects a CS already present in the model's or structure introduced by the projection.

The parameters of these projection functions f_{enc} and f_{dec} are learned by minimizing $\mathcal{L}_{\text{auto}}$ using gradient-based optimization, as introduced in Section 1.4. Under this formulation, each dimension of $\mathcal{Z}$ is interpreted as a dedicated concept direction.

The method proves to be powerful when successfully applied to language models [Templeton et al., 2024]. In these applications, SAE are used to desuperpose the LR of LLM. Once this representation is obtained, it has been shown that many concept vectors exhibit semantic properties. A example is the Golden Gate Bridge concept direction, illustrated in Figure 2.8, which activates systematically whenever the input text or image refers to the Golden Gate Bridge, regardless of the language or modality.

SAEs also provide a way to intervene directly on the LR learned by the model. A concept can be artificially activated by modifying its corresponding feature in the sparse representation z, even when the input contains no reference to it. The modified representation is then projected back into the original activation space, after which the model proceeds with standard next-token generation from h^*:

$$h^* = f_{\text{dec}}(z^*), \quad \text{where} \quad z^* = z + \delta\, e_i,$$

where:

- z is the original sparse representation of the input,
- z^* is the modified sparse representation,
- e_i is the basis vector of $\mathcal{Z}$ associated with the target concept direction,
- δ controls the strength of the artificial activation,
- h^* is the resulting edited representation in the original activation space.

With this method, a reference to the Golden Gate Bridge can be injected into the LLM's internal representation. This causes the model's output to shift toward discussing the Golden Gate Bridge, even when the prompt contains no reference to it. This provides evidence that the learned feature plays a causal role in the model's behavior. Such interventions enable control over specific model behaviors through LS editing.

Despite their empirical success, *Superposition Hypothesis* (Hyp. 2) exhibit several important limitations that question their assumptions. These limitations can be summarized as follows:

- **Instability of the decomposition.** Repeating the SAE training process multiple times on the same model does not yield the same decomposition of the LS. Different runs produce different concept directions in $\mathcal{Z}$, even when achieving comparable reconstruction performance, which raises questions about the reliability of the discovered CS.
- **Lack of semantic coverage.** In the high-dimensional space $\mathcal{Z}$, a large fraction of dimensions do not correspond to any interpretable semantic concept. This abundance of non-semantic directions raises questions about whether the decomposition truly reflects the model's CS.
- **Sparsity–semantics trade-off.** Increasing the sparsity coefficient λ beyond a certain threshold degrades the semantic quality of the learned features [Jermyn et al., 2024]. Suggesting that enforcing excessive sparsity comes at the cost of interpretability.
- **Failure of scaling to remove superposition.** Under the *Superposition Hypothesis* (Hyp. 2), increasing the dimensionality of the representation space should reduce superposition. However, this is not observed in practice. Even in very high-dimensional LS, representations remain superposed, indicating that scaling alone does not resolve the entanglement.

These limitations suggest that *Superposition Hypothesis* (Hyp. 2), while powerful, do not fully resolve the problem of CS. Convincing empirical results do not necessarily validate the underlying theoretical assumptions. Probing (Section 2.2), interpretable directions (Section 2.3), and SAE all rest on a shared assumption that the CS can be recovered through linear decompositions of LR. This assumption has driven significant empirical progress, yet it remains theoretically fragile. The following section therefore introduces a approach that relaxes this constraint.

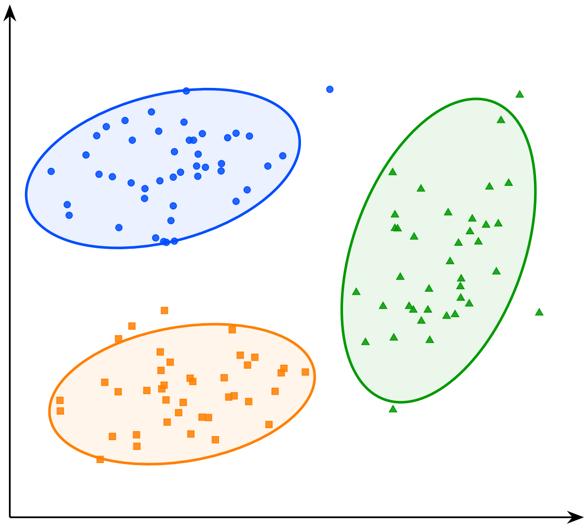


Figure 2.9: Visualization of the latent space structure. The points represent embeddings associated with concepts A, B, and C. The ellipses indicate the approximate extent of each group in the latent space and highlight their geometric separation.

2.5 Clustering

All the LS analysis methods presented so far rely on assumptions about a linear CS.For probing methods, concepts must be at least partially linearly separable for simple probes to succeed 2.2. Latent directions assumes that variations associated with a concept can be captured by moving along a linear axis of the LS 2.3. Similarly, the superposition hypothesis (Hypothesis 2) assumes that concepts are represented through linear directions that become entangled because of representational capacity constraints (Section 2.4). Although these assumptions have led to empirical successes, their theoretical justification remains limited. Several observations suggest that the linearity hypothesis may not fully capture the CS.

For instance, some concepts can only be recovered using non-linear probes. Likewise, methods based on interpretable directions typically identify only a limited number of semantic factors. Finally, in sparse autoencoder approaches, enforcing excessive sparsity often degrades reconstruction quality and interpretability, suggesting that the underlying representation may not be perfectly described by a sparse linear decomposition. Taken together, these observations indicate that semantic concepts may not always exhibit a linear CS. This motivates the exploration of alternative approaches that rely on weaker assumptions regarding the geometry of LS.

To the best of our knowledge, clustering-based analysis is the only family of LS analysis methods that does not rely on any assumption of a linear CS [Ren et al., , Wei et al.,]. Rather than assuming that concepts correspond to directions, clustering methods only assume that conceptual similar inputs produce nearby LR. Conversely, inputs that differ significantly from a concept perspective are expected to be located farther apart in the LS. This is a considerably weaker

Figure 2.10: Clusters discovered by a deep clustering model applied to natural images [Xie et al.,]. Each row contains the ten highest-scoring elements of a single cluster. The visual coherence within each row, grouping animals, vehicles, and aircraft into distinct clusters, suggests that the learned latent representations capture semantically meaningful structure without any explicit supervision.

assumption, in the sense that it is empirically verified in a wide range of settings.

Under this assumption, the discovery of concepts can be reformulated as the identification of groups of embeddings that occupy the same region of the LS. Concepts are therefore no longer associated with directions, but with clusters of neighboring representations, as illustrated in Figure 2.10. This idea forms the basis of the research field known as deep clustering.

In these approaches, a collection of inputs is projected into a LS using a NN. The resulting embeddings form a point cloud on which standard clustering algorithms can be applied. The obtained clusters are then interpreted as representing distinct concepts.

Deep clustering has been particularly successful in unsupervised image classification. Typically, an autoencoder is composed of an encoder f_{enc} and a decoder f_{dec}. The encoder maps an input $x^{(i)} \in \mathcal{X}$ to a low-dimensional LR $z^{(i)} \in \mathbb{R}^d$:

$$z^{(i)} = f_{\text{enc}}(x^{(i)}), \quad d \ll \dim(\mathcal{X}).$$

To ensure that this compressed representation does not discard essential information, the decoder reconstructs the input from the LS:

$$\hat{x}^{(i)} = f_{\text{dec}}(z^{(i)}) = f_{\text{dec}}(f_{\text{enc}}(x^{(i)})).$$

The model is trained to minimize the reconstruction error between the original input and its reconstruction, typically using a Mean Squared Error (MSE) loss:

$$\mathcal{L}_{\text{rec}} = \frac{1}{N} \sum_{i=1}^{N} \left\| x^{(i)} - \hat{x}^{(i)} \right\|_2^2.$$

This objective encourages the LS to preserve the most informative features of the input while enforcing a compact representation.

Clustering algorithms are then applied to the learned embeddings.

Let $\mathcal{D} = \{x^{(i)}\}_{i=1}^{N}$ denote a dataset of inputs. Through the encoder f_{enc}, this dataset induces a point cloud in the LS:

$$\mathcal{Z} = \{z^{(i)} \in \mathbb{R}^d \mid z^{(i)} = f_{\text{enc}}(x^{(i)}),\ x^{(i)} \in \mathcal{D}\}.$$

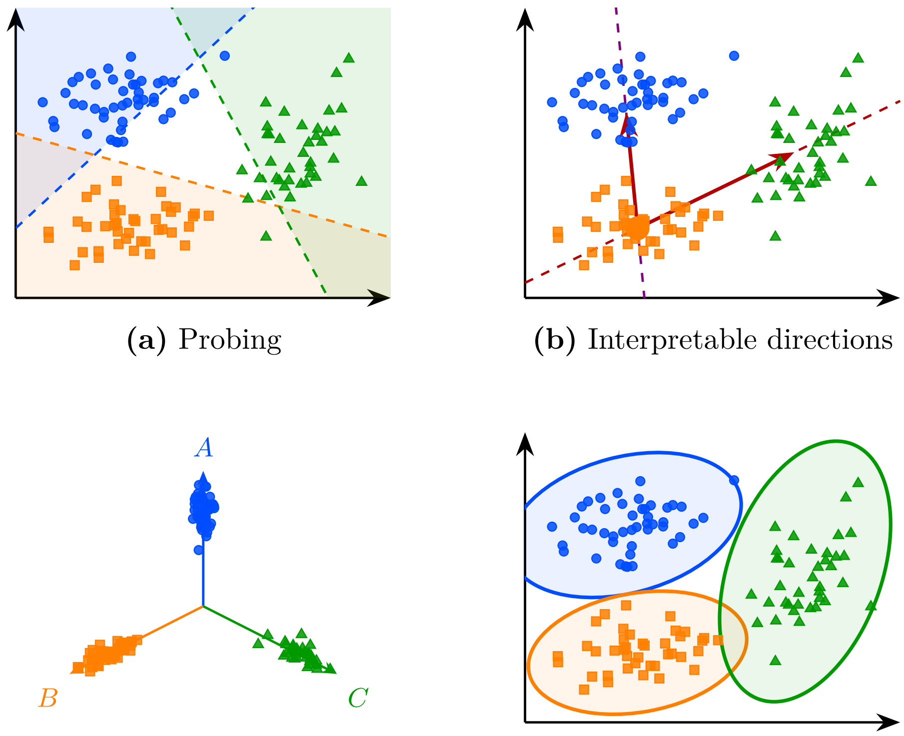
Clustering algorithms are then applied to this set $\mathcal{Z}$ of embeddings in order to identify groups of similar representations.

An example of this methodology can be observed in Figure 2.10, where the approach is applied to the STL-10 dataset [Coates et al., 2011] using k-means clustering [MacQueen, 1967] with a number of clusters arbitrarily fixed to $K = 10$ [Xie et al.,]. For each cluster, the corresponding line shows the 10 samples closest to the associated centroid in the LS.

We observe that the method captures coherent semantic categories such as animals, vehicles, and flying entities. This result is particularly striking given that the only available information to the system consists of raw images. It suggests that the compression process induces a LS in which semantically meaningful CSs emerge.

More advanced approaches combine representation learning and clustering objectives within a single optimization process. In this setting, additional loss terms encourage embeddings to organize themselves around cluster centroids during training. This often produces LS that are easier to analyze and improves the quality of the discovered semantic CS [Guo et al., , Miklautz et al.,].

Despite its effectiveness, clustering-based analysis presents several important limitations. First, as with most unsupervised semantic discovery techniques, interpreting the resulting clusters remains partly subjective. Benchmark datasets often provide class labels for evaluation, but the discovered clusters may not match the categories that humans consider relevant. Alternative semantic groupings may also be equally valid. Second, clustering methods are highly sensitive to design choices, as the number of clusters, the distance metric, and the clustering algorithm itself can all significantly influence the results, making parameter selection challenging and often requiring substantial empirical tuning. Finally, to the best of our knowledge, no intervention technique has yet been proposed in the context of clustering-based analysis. Unlike probing or SAE analysis, which allow targeted modifications of LR to study their causal role, clustering methods currently offer no mechanism to alter the model's behavior by acting directly on the discovered CS.



(a) Probing **(b)** Interpretable directions

(c) Sparse autoencoder **(d)** Clustering

Figure 2.11: Overview of the four latent space analysis methods studied in this chapter, applied to the same point cloud.

2.6 Conclusion

The four families of methods reviewed in this chapter, namely probing, interpretable directions, sparse autoencoders, and clustering, are summarized in Figure 2.11 and all build upon the *Concept Hypothesis* (Hyp. 1) introduced in Section 2.1. Each of them approaches the question from a different angle, but they share a common objective: uncover how LR are structured and what information can be extracted from them.

However, a fundamental difficulty remains. There is currently no consensus on how to rigorously evaluate to what extent these methods genuinely provide access to the internal concepts of a model. Most existing metrics assess reconstruction quality or linear separability, but these do not directly measure whether the discovered CSs correspond to meaningful abstractions from the model's perspective. This leaves open the question of what it means, formally, for a concept to be represented in a NN.

Addressing this question may require contributions from disciplines beyond computer science and mathematics. As the notion of semantics is inherently tied to human interpretation, a purely formal or computational account may be insufficient. Insights from philosophy, cognitive science, or psychology could prove valuable in grounding the notion of concept in a way that is both theoretically rigorous and empirically tractable. Despite these open questions, the methods studied in this chapter provide a practical foundation for analyzing the internal representations of NN.

Chapter 3

Reinforcement Learning

This chapter introduces the fundamental concepts of Reinforcement Learning (RL), providing the necessary foundation for the ideas developed throughout the rest of this manuscript. Its structure is as follows:

Section 3.1: An overview of the milestones that led to the emergence of RL as a unified field.

Section 3.2: A formalization of the objective underlying RL algorithms.

Section 3.3: An explanation of how policies are progressively refined through interaction with the environment to approach the optimal policy.

3.1 Historical Developments

RL is today presented as a major field within artificial intelligence. However, it originates from several research traditions that progressively converged toward a common framework. In this section, we outline the main stages in the emergence of this field, highlighting the key ideas and milestones that shaped its evolution.

3.1.1 Behavioral foundations (1900–1950)

The origins of RL can be traced back to 20th-century animal psychology studies [Thorndike, 1911]. These studies aimed to understand how behaviors are formed and modified. The central idea emerging from this work is that learning is driven by the consequences of actions. Behaviors followed by positive outcomes are more likely to be repeated, whereas those associated with negative outcomes tend to gradually decrease in frequency.

This perspective was later developed within behaviorism. In this school of psychology, rewards and punishments are used to gradually shape behavior through repeated interaction with the environment [Skinner, 1938]. The term "reinforcement" in the context of learning comes from this school of thought, where it refers precisely to the process of strengthening a behavior through its consequences.

This idea led to the suggestion that learning principles observed in living organisms could be transferred to artificial systems. Machines were therefore

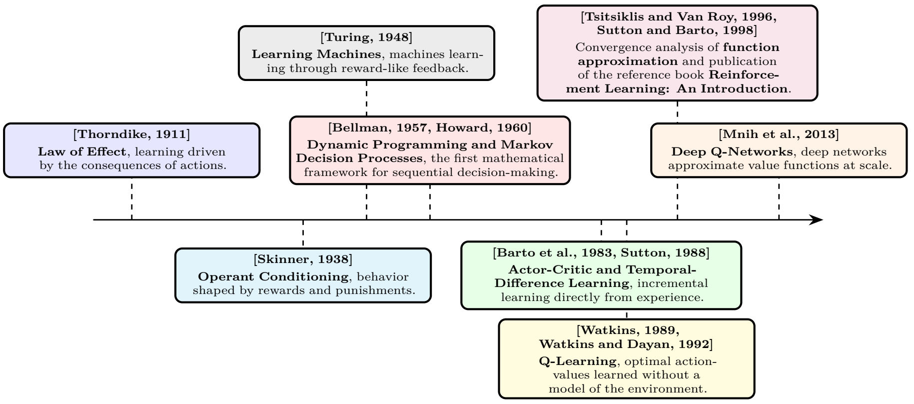


Figure 3.1: Evolution of reinforcement learning prior to its fusion with deep learning.

envisioned as potentially learning through reward-like mechanisms, guided by evaluative feedback signals resembling a "pleasure–pain system" [Turing, 1948]. Although these studies did not involve formal mathematical models, they introduced the central idea of RL, trial-and-error learning guided by environmental feedback.

3.1.2 Sequential decision-making (1950–1970)

A second foundation of RL comes from optimal control theory. This field studies how to determine sequences of decisions that optimize a cumulative performance criterion over time. In this context, the decision-making entity is called an agent. The agent interacts with the environment by selecting actions. The function that maps an observed state to the chosen action is called a policy. The objective is to find an optimal policy, in the sense that it maximizes the expected cumulative reward over time.

This framework led to the formalization of Markov Decision Processes (MDP), which provide a mathematical model for sequential decision-making [Howard, 1960]. MDP describe a decision-maker interacting with an environment through a set of states, actions, rewards, and transition rules. The earliest method proposed to solve MDP is dynamic programming [Bellman, 1957]. It proceeds by breaking down a complex decision problem into smaller, more manageable subproblems, solving each one, and combining the results to obtain an overall solution. However, dynamic programming requires a full and exact description of the environment, which is only feasible for small problems.

3.1.3 Emergence of reinforcement learning (1970–2000)

In order to address this issue, temporal-difference learning methods were introduced [Sutton, 1988]. This mechanism enables incremental learning directly from experience without requiring a complete model of the environment. It can be interpreted as a sample-based approximation of the Bellman equations.

Building on these ideas, the notion of value functions emerged as a way to estimate the long-term benefit of being in a given state or taking a given action. These functions assign a numerical score to states or state-action pairs, reflecting the expected cumulative reward an agent can obtain from that point onward. They thus provide a compact way to evaluate and compare different courses of action. This line of work culminated in Q-learning, which showed that optimal action-value functions can be learned directly from interaction with the environment, without requiring a model of its dynamics [Watkins, 1989, Watkins and Dayan, 1992].

The publication of the foundational reference work [Sutton and Barto, 1998] contributed to the formalization of RL as a coherent field. From that point onward, the field can be considered well established and widely recognized as a distinct area of research.

3.1.4 Consolidation of reinforcement learning (2000–2013)

A limitation of early algorithms such as Q-learning is their reliance on explicit state–action tables. Although these methods remove the need to know the full dynamics of the MDP, they still require storing a value for each possible state–action pair. As a result, the agent must effectively visit and maintain estimates for a potentially enormous number of states in the environment. This requirement becomes infeasible in large-scale MDP, where the state space is too large to be exhaustively explored or stored in memory.

To address this issue, research shifted toward more general representations of value functions and policies. Instead of maintaining explicit tables, function approximation methods were introduced. These allowed value functions and policies to be represented using parametric models such as linear approximators or perceptron-like architectures [Barto et al., 1983, Tsitsiklis and Van Roy, 1996]. This change enabled generalization across states, reducing the dependence on exhaustive exploration of the state space. The introduction of Deep Q-Networks (DQN), which demonstrated that deep Neural Network (NN) could be used to approximate value functions at scale [Mnih et al., 2013].

Figure 3.1 provides a timeline of the key milestones that shaped the development of reinforcement learning prior to its fusion with deep learning.

3.1.5 Deep reinforcement learning (2013–2026)

The introduction of Deep Learning (DL) into RL marked a major turning point in the field. A series of algorithms significantly improved scalability. Deep Deterministic Policy Gradient extended actor–critic methods to continuous action spaces [Lillicrap et al., 2015]. Proximal Policy Optimization (PPO) later became a standard algorithm in RL due to its simplicity, stability, and ability to perform in both continuous and discrete action spaces [Schulman et al., 2017].

These advances established a practical foundation for large-scale policy optimization. Notably, AlphaZero demonstrated that the combination of deep NN, RL, and tree search could reach superhuman performance in the game of Go [Silver et al., 2017b]. This success was later extended to even more complex real-time strategy environments such as StarCraft II, where agents must handle long-term planning, partial observability, and large action spaces [Samvelyan et al., 2019]. Similarly, in Dota 2, large-scale multi-agent RL systems such as OpenAI Five showed that self-play combined with deep policy

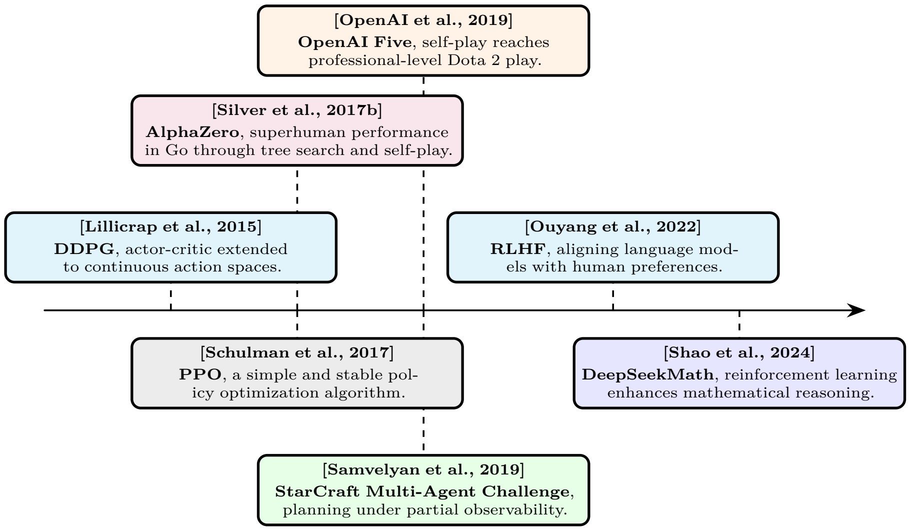


Figure 3.2: Evolution of deep reinforcement learning.

optimization can achieve professional-level performance in highly dynamic and stochastic environments [OpenAI et al., 2019].

More recently, RL has expanded beyond its traditional application domains. With the emergence of increasingly agentic views of Large Language Model (LLM), it has become possible to train these models to perform specific tasks using RL-based techniques. In particular, RL from human feedback has become a method for aligning LLM with human preferences. It does so by formalizing the problem as an MDP and using RL to solve it [Ouyang et al., 2022]. Similarly, DeepSeekMath shows how RL can enhance the mathematical reasoning abilities of LLM [Shao et al., 2024]. It demonstrates that an artificial agent can learn to reason in natural language in order to accomplish a given task. The skills acquired during this learning process also transfer to other similar tasks, leading to improved overall performance. For instance, training a LLM with RL on mathematical problem-solving can also enhance its programming capabilities.

Originally inspired by biological learning processes, RL is now closer than ever to this initial intuition. Agents learn through interaction, adapt via trial and error, and progressively improve their behavior based on experience. This behavior is governed by deep neural architectures, which are the closest computational analog to organic brains. More than ever, RL stands as one of the core paradigms for achieving artificial general intelligence [Silver et al., 2021].

Figure 3.2 provides a timeline of the key milestones that shaped the emergence of deep reinforcement learning.

3.2 Objective

Now that the main developments shaping RL have been outlined, the formal definition of its objective can be introduced. In essence, RL aims to build a system that makes sequential decisions in order to maximize a cumulative reward signal over the long term. The purpose of this section is to translate this intuitive goal into a rigorous mathematical framework. To this end, the presentation proceeds step by step:

1. Modeling the reinforcement learning problem as a Markov Decision Process.
2. Defining the notion of a policy, which formalizes the agent's behavior.
3. Describing the interaction between the agent and the environment through trajectories.
4. Introducing value functions to evaluate policies.
5. Defining the notion of an optimal policy.

3.2.1 Markov Decision Processes

The RL framework is grounded in the agent paradigm: the decision-making entity is called the agent. The agent perceives its environment, processes the information it receives, and acts upon the environment to influence future outcomes. To specify the task and the agent's capacity to perceive and act, the problem is modeled as a MDP.

Before providing a more detailed definition, it is useful to clarify the scope of the MDP problems considered in this manuscript. We therefore restrict the analysis to the following configuration:

- The single-agent learning setting is considered. If any other agents are present in the environment, they are treated as part of the environment dynamics.
- Finite-horizon MDP are considered. Each interaction sequence between the MDP and the policy has a defined beginning and a defined end. Between these two points, the number of interaction steps may vary depending on the situation, but it always remains finite.
- The partially observable framework is adopted, where the agent does not have direct access to the true state of the system, but instead relies on limited observations that provide only partial information about the environment.

The resulting framework can be formally described as a single-agent finite-horizon partially observable Markov decision process. For simplicity, we will refer to this setting as an "MDP" throughout this manuscript.

The following section aims to provide a mathematical description of the RL problem. The formulation adopted in this manuscript has been chosen to emphasize the main elements that will be used throughout the remainder of the manuscript. A MDP is defined by the tuple $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}, p)$:

- $\mathcal{S}$: The set of states, representing all possible configurations of the environment. A state belonging to this set is denoted by $s \in \mathcal{S}$. Among these, two special subsets are distinguished: initial states, from which an interaction can begin, and terminal states, in which the interaction ends. The set of initial states is denoted by $\mathcal{S}^i$, with an initial state written as $s^i \in \mathcal{S}^i$. The set of terminal states is denoted by $\mathcal{S}^t$, with a terminal state written as $s^t \in \mathcal{S}^t$.

- $\mathcal{O}$: The set of observations that the agent can perceive from the environment. A observation belonging to this set is denoted by $o \in \mathcal{O}$. Unlike the true state $s \in \mathcal{S}$, which is hidden from the agent, the observation $o \in \mathcal{O}$ provides partial information about the current state.
- $\mathcal{A}$: The set of actions that the agent may execute. A action belonging to this set is denoted by $a \in \mathcal{A}$. Actions influence the dynamics of the MDP, thereby shaping its future course. They represent the agent's capacity to act upon its environment.
- $\mathcal{R}$: The set of rewards that the agent can receive from the environment. A reward belonging to this set is denoted by $r \in \mathcal{R}$. $\mathcal{R}$ is a subset of $\mathbb{R}$. Rewards can be either positive or negative, depending on whether the corresponding outcome is considered beneficial or detrimental for the agent. The agent's goal is to maximize the cumulative sum of these rewards over time.
- p: The transition function, which defines the dynamics of the environment. Given a state s and an action a, this function defines a probability distribution over the joint outcome consisting of the next state s', the observation o, and the reward r. This function is denoted by p, and is defined as: $p : \mathcal{S} \times \mathcal{A} \to \mathcal{P}(\mathcal{S} \times \mathcal{O} \times \mathcal{R})$. The notation $(s', o, r) \sim p(s, a)$ means that the tuple (s', o, r) is sampled from the distribution induced by $p(\cdot \mid s, a)$. The notation $p(s', o, r \mid s, a)$ denotes the probability of observing the tuple (s', o, r) under the distribution $p(\cdot \mid s, a)$.

The two quantities, observation o and state s, are not independent. They are jointly generated via $(s', o, r) \sim p(s, a)$, which implicitly defines the relationship between them. In our MDP formalization, this relationship is left implicit because it is never manipulated directly during learning.

However, to express mathematical equations, it is convenient to introduce the conditional distribution over states given an observation. We denote this distribution by $b : \mathcal{O} \to \mathcal{P}(\mathcal{S})$. The notation $b(s \mid o)$ denotes the probability of state s given the observation o, while $s \sim b(\cdot \mid o)$ indicates that the state s is sampled from this distribution.

3.2.2 Policy

The agent's decision-making capability is modeled by a policy function. The policy defines the stochastic mapping between observations $o \in \mathcal{O}$ and actions $a \in \mathcal{A}$. A policy function is denoted by π, and is defined as: $\pi : \mathcal{O} \to \mathcal{P}(\mathcal{A})$. The notation $a \sim \pi(\cdot \mid o)$ means that the action a is sampled from the distribution induced by $\pi(\cdot \mid o)$. The notation $\pi(a \mid o)$ denotes the probability of selecting action a under the distribution $\pi(\cdot \mid o)$.

It is important to note that this policy does not include any explicit state-estimation mechanism. Instead, the ability to infer information about the underlying state is entirely left to the policy itself, which must learn from experience how to compensate for the missing information. This reflects the design of many modern RL implementations, where policies are often parameterized by architectures capable of summarizing past observations, such as recurrent or attention-based networks, without explicitly modeling the uncertainty over the underlying state.

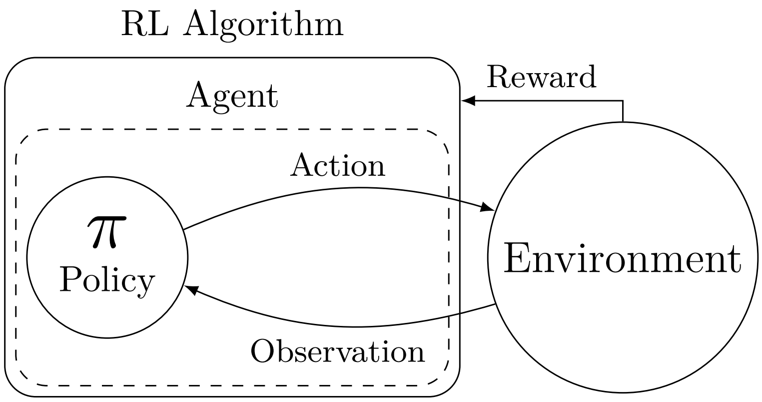


Figure 3.3: Reinforcement learning loop.

Now that the MDP and the policy have both been defined, Figure 3.3 illustrates how they interact within the general RL loop. As a reminder, the objective of RL is to iteratively improve the policy so as to maximize the reward function.

3.2.3 Trajectory

Now that both the MDP $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}, p)$ and the policy π have been defined, the interaction between them can be characterized. A sequence of consecutive interactions between an environment and a policy is called a trajectory. A trajectory is generated through the repeated interaction between the policy and the environment, following the sequence below:

1. The environment, being in state s, provides an observation o to the policy π. Based on this observation, the policy outputs a probability distribution over actions $\pi(\cdot \mid o)$, from which an action is sampled: $a \sim \pi(\cdot \mid o)$.

2. The sampled action is executed in the environment. The environment then generates a transition: $(s', o', r) \sim p(\cdot \mid s, a)$. It moves to a new state s', and returns a new observation o' and a reward r.

A trajectory t of length T can be written as:

$$t = (s_1, o_1, a_1, r_1, s_2, o_2, a_2, r_2, \ldots, s_T, o_T, a_T, r_T).$$

It is important to clearly distinguish between a trajectory distribution and its realizations. The distribution over trajectories is denoted by τ, and a particular trajectory is denoted by t. The distribution τ is determined by the initial state s, the initial observation o, the environment dynamics p, and the policy π, and is denoted by $\tau(s, o, p, \pi)$. The notation $t \sim \tau(s, o, p, \pi)$ means that the trajectory t is sampled from the distribution induced by s, o, p, and π. The notation $\tau(t \mid s, o, p, \pi)$ denotes the probability of observing the trajectorie t under the distribution $\tau(\cdot \mid s, o, p, \pi)$

An important property of the trajectory distribution is its recursive structure. Since both the policy π and the transition function p are known, the distribution over trajectories starting from s_1 with observation o_1 can be expressed in terms of the distribution over trajectories starting from the next state s_2 with observation o_2. Specifically, the probability of a trajectory t factorizes as:

$$\tau(t \mid s_1, o_1, p, \pi) = \pi(a_1 \mid o_1)\, p(s_2, o_2, r_1 \mid s_1, a_1)\, \tau(t_{2:} \mid s_2, o_2, p, \pi),$$

where $t_{2:} = (s_2, o_2, a_2, r_2, \dots, s_T, o_T, a_T, r_T)$ denotes the trajectory starting from the second step.

There exist two special types of trajectories. First, a trajectory that terminates in a terminal state is called a terminal trajectory, and is denoted by $t_t \sim \tau_t(s, o, p, \pi)$. Second, a terminal trajectory that starts from an initial state and an initial observation is called an episode, and is denoted by $t_e \sim \tau_t(s^i, o, p, \pi)$.

3.2.4 Value Functions

To evaluate the ability of a policy π to collect rewards, the notion of return is introduced. The return of a trajectory $g(t, \gamma)$, is defined as the discounted sum of rewards collected along the trajectory:

$$g(t, \gamma) = r_1 + \gamma r_2 + \gamma^2 r_3 + \cdots + \gamma^{T-1} r_T = \sum_{i=1}^{T} \gamma^{i-1} r_i,$$

where $\gamma \in [0, 1]$ is the discount factor. This parameter controls the time horizon of the agent's objective. When γ is close to 0, the agent focuses almost exclusively on immediate rewards, leading to short-sighted behavior. As γ increases toward 1, future rewards gain more weight, and the agent adopts a more far-sighted strategy. In theory, since the goal is to maximize long-term cumulative reward, the discount factor would ideally be set to $\gamma = 1$. In practice, however, γ is often chosen slightly below 1 (e.g., 0.99) to stabilize learning, and to reflect the fact that distant future rewards are inherently more uncertain.

The return also admits a recursive form, which is central to the dynamic programming and RL algorithms presented later:

$$g(t, \gamma) = r_1 + \gamma\, g(t_{2:}, \gamma),$$

From the notion of return, functions that estimate the expected return of a policy from a given starting point can be defined. These functions are called value functions. The first one considered is the state-value function V. It tells us how desirable a particular state is under a given policy π, given an observation o, the environment dynamics p, and the discount factor γ. Strictly speaking, this function should be written as $V(s, o, p, \gamma, \pi)$ to reflect all its dependencies. However, since both the environment dynamics p and the discount factor γ are fixed throughout learning, they are omitted from the notation. To emphasize that the value is tied to a specific policy, π is placed as a subscript. This yields the compact notation $V_\pi : \mathcal{S} \times \mathcal{O} \to \mathbb{R}$.

If $V_\pi(s, o) > V_\pi(s', o')$, then, under policy π, the agent prefers being in state s with observation o over state s' with observation o'. Formally, the state-value function is defined as:

$$V_\pi(s, o) = \mathbb{E}\Big[g(t, \gamma)\Big], \quad \text{with } t \sim \tau_t(s, o, p, \pi).$$

It is worth noting that the state-value function V_π is recursive, a property that follows directly from the recursive form of the return:

$$\begin{aligned} V_\pi(s,o) &= \mathbb{E}\Big[g(t,\gamma)\Big], \quad \text{with } t \sim \tau_t(s,o,p,\pi) \\ &= \mathbb{E}\Big[r_1 + \gamma\, g(t_{2:},\gamma)\Big] \\ &= \mathbb{E}\Big[r_1 + \gamma\, V_\pi(s_2)\Big] \end{aligned}$$

If the expectation is expressed explicitly in terms of the distributions p and π, the Bellman equation for V_π is obtained:

$$\begin{aligned} V_\pi(s,o) &= \mathbb{E}\Big[r_1 + \gamma\, V_\pi(s_2)\Big] \\ &= \sum_{a\in\mathcal{A}} \pi(a \mid o) \sum_{\substack{s'\in\mathcal{S}\\ o'\in\mathcal{O}\\ r\in\mathcal{R}}} p(s',o',r \mid s,a)\Big[r + \gamma\, V_\pi(s',o')\Big]. \end{aligned}$$

Now that the value of a state has been expressed, the next step is to consider the value of taking a specific action in a given state. Indeed, it is useful to estimate the expected future gain of an action a in state s, in order to compare different actions and determine which one is the most promising under a given policy π. This function is called the state-action value function $Q_\pi : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$. If $Q_\pi(s,a) > Q_\pi(s,a')$, then, under policy π, choosing action a in state s yields a higher expected return than choosing action a'. It is defined as:

$$Q_\pi(s,a) = \mathbb{E}\Big[\, r + \gamma\, g(t,\gamma)\,\Big], \quad \text{with } t \sim \tau_t(s',o,p,\pi), \quad (s',o,r) \sim p(\cdot \mid s,a).$$

The function Q_π can be related to V_π by recognizing that, after taking action a in state s, the remaining return from the next state s' is precisely $V_\pi(s')$. This yields:

$$\begin{aligned} Q_\pi(s,a) &= \mathbb{E}\Big[\, r + \gamma\, g(t,\gamma)\,\Big], \quad \text{with } t \sim \tau_t(s',o,p,\pi), \quad (s',o,r) \sim p(\cdot \mid s,a) \\ &= \mathbb{E}\Big[\, r + \gamma\, V_\pi(s',o)\,\Big] \\ &= \sum_{\substack{s'\in\mathcal{S}\\ o'\in\mathcal{O}\\ r\in\mathcal{R}}} p(s',o',r \mid s,a)\Big[\, r + \gamma\, V_\pi(s',o)\,\Big] \end{aligned}$$

3.2.5 Optimal Policy

The goal of RL is to find a policy that maximizes the expected cumulative reward from any observation until the end of the trajectory. This policy is called the optimal policy and is denoted by π^*. In a partially observable setting, the optimal action is the one that maximizes the expected state-action value over the distribution of possible states given the observation:

$$\pi^*(a \mid o) = \begin{cases} 1 & \text{if } a = \underset{a'\in\mathcal{A}}{\arg\max}\ \sum_{s\in\mathcal{S}} b(s \mid o)\, Q_{\pi^*}(s,a'), \\ 0 & \text{otherwise} \end{cases}$$

As shown, both V_π and Q_π can be expressed explicitly in terms of the environment dynamics p and the policy π. If p, b and π are fully known, one can theoretically compute the optimal policy by unfolding the graph of all possible trajectories starting from a given state s and observation o. This is possible precisely because V_π and Q_π are recursive functions. Solving an MDP in this way is known as dynamic programming.

In practice, most environments of have too many states to allow an explicit representation of the dynamics p. Storing and computing over the full transition function becomes intractable. To overcome this limitation, instead of directly constructing the optimal policy, an iterative approach is adopted: starting from an initial policy, it is progressively improved so that it converges toward π^*. This iterative improvement through interaction with the environment is precisely what constitutes learning in RL, and it is the subject of the next section.

3.3 Learning

Now that the optimal policy π^* has been defined, the practical question of how to obtain it arises. The recursive Bellman equations for V_π and Q_π suggest dynamic programming approaches, but these scale poorly to large state spaces. RL offers an alternative: it seeks to solve the MDP iteratively, by generating a sequence of policies that progressively approach π^* through direct interaction with the environment. In this section, two major families of RL methods used today are introduced: critic-only methods and actor-critic methods.

3.3.1 Critic-Only Methods

The central idea behind all critic-only approaches is to iteratively construct an approximation of the state-action value function Q_π, denoted by $\hat{Q}_\pi$. Although $\hat{Q}_\pi$ is only an approximation of the true Q_π, the policy always acts greedily with respect to this approximation:

$$\pi(a \mid o) = \begin{cases} 1 & \text{if } a = \arg\max_{a' \in \mathcal{A}} \hat{Q}_\pi(o, a'), \\ 0 & \text{otherwise.} \end{cases} \tag{3.1}$$

Since $\hat{Q}_\pi$ serves as the basis for the agent's decisions, it cannot rely on more information than the agent itself has access to. It must therefore be defined solely over observations, as a function $\hat{Q}_\pi : \mathcal{O} \times \mathcal{A} \to \mathbb{R}$. Its objective is to approximate the true Q_π, which depends on the underlying states s, using only the partial information available through o. Once again, this poses no issue in practice, as it reflects how these algorithms are actually deployed in real-world applications.

It is precisely this way of modeling the policy that gives this family of methods its name. The policy is not represented explicitly as a standalone function. Rather, it simply emerges from searching over the set of actions to maximize $\hat{Q}_\pi$.

To give a clear example of critic-only methods, they are presented in their simplest form: model-free, without temporal-difference learning, without replay buffer, without bootstrapping, and using only ε-greedy exploration. Naturally, many variants and extensions of this basic scheme exist, but the core idea remains the same.

When implementing a RL algorithm, one of the first questions is how to represent the value function. In critic-only methods, this means choosing a model for $\hat{Q}_\pi$. Any machine learning model capable of regression task can be used for this purpose. The set of parameters of the model is denoted by θ. In deep RL, a NN is used to model this function, in which case θ represents the weights and biases of the network.

In our simple setting, exploration is handled via ε-greedy selection. With probability $1-\varepsilon$, the policy selects the greedy action that maximizes $\hat{Q}_\pi$, and with probability ε it selects an action uniformly at random. Formally, this ε-greedy policy is obtained by modifying the greedy policy defined above as follows:

$$\pi(a \mid o) = \begin{cases} 1 - \varepsilon + \dfrac{\varepsilon}{|\mathcal{A}|} & \text{if } a = \underset{a' \in \mathcal{A}}{\arg\max}\, \hat{Q}_\pi(o, a'), \\ \dfrac{\varepsilon}{|\mathcal{A}|} & \text{otherwise.} \end{cases} \tag{3.2}$$

The trajectory distribution induced by the resulting policy now depends on both π and ε, and is denoted by $t \sim \tau(s, o, p, \pi, \varepsilon)$. The parameter ε controls the degree of exploration. If the policy always chooses the action it currently believes to be optimal, it may miss better alternatives that $\hat{Q}_\pi$ has not yet learned to value correctly. Exploring suboptimal actions from time to time is therefore essential to verify that $\hat{Q}_\pi$ accurately reflects the true values, while still focusing most of the time on the most promising actions. In more advanced implementations, ε can be adapted over the course of learning to control exploration dynamically, for instance by decreasing it gradually over time.

The learning process consists of two steps that repeat until convergence:

1. **Data collection.** The current policy π interacts with the MDP. For a given starting state s_1^i and observation o_1, the interaction produces a episode t_e sampled from $\tau_t(s_1^i, o_1, p, \pi, \varepsilon)$. For each step i along the episode, the return from that step onward is $g(t_{i:}) = \sum_{k=i}^{T} \gamma^{k-i} r_k$, which serves as the target value for the state-action pair (s_i, a_i). This procedure is detailed in Algorithm 2.

2. **Model update.** The tuples $(s_i, a_i, g(t_{i:}))$ obtained in the previous step serve as supervised training data: the state-action pair (s_i, a_i) is the input, and the return $g(t_{i:})$ is the associated target. These examples are used to train $\hat{Q}_\theta$. The improved approximation is denoted by $\hat{Q}'_\theta$, which now predicts returns more accurately under π. Since $\hat{Q}'_\theta$ has changed, the policy derived from it also changes. This new policy is denoted by π'.

Because the policy is now π', the approximation $\hat{Q}'_\theta$ must be trained to predict returns under π'. This requires returning to step 1 to collect fresh interaction data generated by π'. The two steps thus alternate until the policy and the value approximation stabilize. Algorithm 3 summarizes this whole critic-only learning loop.

Even this simple algorithm already illustrates a difficulty inherent to all forms of RL: the instability of the learning process. Each update from $\hat{Q}_\theta$ to $\hat{Q}'_\theta$ causes the policy to shift from π to π'. For the learning to remain stable, the change from $\hat{Q}_\theta$ to $\hat{Q}'_\theta$ should be as small as possible. This way, $\hat{Q}'_\theta$, trained to predict

Algorithm 2: Data Collection

Input:
MDP $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}, p)$
Policy π
Discount factor $\gamma \in [0, 1]$
Output:
Dataset $\mathcal{D}$

Sample an initial state.
$i \leftarrow 1$;
Sample $s_1 \in \mathcal{S}^i$;
Observe o_1;

Episode generation by interaction with the MDP.
repeat
    Action selection.
    Sample $a_i \sim \pi(\cdot \mid o_i)$;
    Environment transition.
    Sample $(s_{i+1}, o_{i+1}, r_i) \sim p(\cdot \mid s_i, a_i)$;
    $i \leftarrow i + 1$;
until $s_i \in \mathcal{S}^t$;

Return computation for each step of the trajectory.
$T \leftarrow i - 1$;
for $i = 1, \ldots, T$ **do**
    $g_i \leftarrow \sum_{k=i}^{T} \gamma^{k-i} r_k$;

Dataset construction
$\mathcal{D} \leftarrow \left\{ \left(s_i, o_i, a_i, g_i \right) \right\}_{i=1}^{T}$;
Return $\mathcal{D}$

returns under π, remains reasonably accurate under π'. However, the smaller the change, the slower the learning progresses. A trade-off must therefore be struck between stability and learning speed. Many improvements introduced later in the literature aim at stabilizing this learning loop [Hessel et al., 2017].

Convergence guarantees for such algorithms have been proved under a different set of assumptions than the setting presented here. For example, in the classical convergence results, the MDP is fully observable, the value function is stored in a table, and the learning rate is decreased at a suitable rate over time [Watkins and Dayan, 1992]. In modern implementations, the value function is approximated by a NN. Theoretical convergence proofs no longer hold in this setting. Nevertheless, deep methods have demonstrated remarkable empirical success, making them the tool of choice for RL tasks.

Critic-only approaches suffer from several limitations. The main limitation is scalability to large action spaces. Since the policy must evaluate all possible actions at each decision step to select the maximum, the computational cost grows with the size of the action set. To address this issue, another family of RL algorithms exists: actor-critic methods.

Algorithm 3: Deep Critic-Only Reinforcement Learning

Input:
MDP $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}, p)$
Architecture $\hat{Q}$
Exploration rate $\varepsilon \in [0, 1]$
Discount factor $\gamma \in [0, 1]$
Learning rate $\eta > 0$
Initial parameters θ (optional)
Output:
Trained policy π_θ

```
if θ is not given then
    Initialize θ randomly;
// Exploration policy as defined in Equation (3.2)
π_θ ← ε-greedy policy with respect to Q̂_θ
// MSE loss function as defined in Equation (1.1)
ℒ ← Mean Squared Error (MSE);
while stopping criterion not satisfied do
    // Data collection see Algorithm 2
    𝒟 ← DataCollection(MDP, π_θ, γ)
    // observation-action pairs and their returns, from 𝒟
    𝒟′ ← {((o, a), g) | (s, o, a, g) ∈ 𝒟}
    // Model update whith SGD see Algorithm 1
    θ ← SGD(𝒟′, Q̂, ℒ, η, θ)
// Final policy without exploration as defined in Equation (3.1)
π_θ ← policy with respect to Q̂_θ;
Return π_θ
```

3.3.2 Actor-Critic Methods

Two principal modifications distinguish actor-critic methods from critic-only approaches. First, the objective is to approximate the state-value function V, instead of the state-action value function Q. Second, the policy π is given its own dedicated set of parameters. Two separate components are thus obtained: a value function approximation $\hat{V}_{\theta_1}$ and a parameterized policy π_{θ_2}.

Analogously to critic-only methods, $\hat{V}_{\theta_1}$ is trained using supervised learning on pairs obtained from interaction with the MDP. For each step i along a trajectory, the input is the state-observation pair (s_i, o_i) and the target is the observed return $g(t_{i:})$. The value function is thus trained to correct its predictions based on data collected by the current policy interacting with the environment.

The key difference lies in how the policy π itself is optimized. Since the policy now has its own set of parameters θ_2, it can be updated directly. The update uses the notion of advantage $A : \mathcal{S} \times \mathcal{A} \to \mathcal{R}$, defined as:

$$A(s, a) = g(t) - \hat{V}_{\theta_1}(s, o),$$

where t is the trajectory that starts by taking action a in state s and then follows π for all subsequent steps.

When $A(s, a) > 0$, the action a performed better than expected, and the policy parameters θ_2 are adjusted to increase the probability $\pi_{\theta_2}(a \mid o)$. When $A(s, a) < 0$, the action a performed worse than expected, and the probability is decreased.

Both $\hat{V}_{\theta_1}$ and π_{θ_2} are improved simultaneously within the same loop: the critic provides a learned baseline for the actor, and the actor generates the data used to train the critic. This interplay reduces the instability inherent to RL. Algorithm 4 summarizes this whole actor-critic learning loop.

PPO (Proximal Policy Optimization), which is the current standard in RL, belongs to this family of actor-critic methods. It will be used throughout this manuscript for all RL experiments.

Now that highly performant policies can be trained, learning purely through interaction with their environment without any human supervision, the question arises of how to explain their behavior. This is the subject of the next chapter: explainability in the context of RL.

Algorithm 4: Deep Actor-Critic Reinforcement Learning

Input:
MDP $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}, p)$
Architecture $\hat{V}$ (critic)
Architecture π (actor)
Discount factor $\gamma \in [0, 1]$
Learning rate $\eta > 0$
Initial parameters θ_1 of the critic (optional)
Initial parameters θ_2 of the actor (optional)
Output:
Trained policy π_{θ_2}

---

if θ_1 *is not given* **then**
  Initialize θ_1 randomly;
if θ_2 *is not given* **then**
  Initialize θ_2 randomly;

MSE loss function as defined in Equation (1.1)
$\mathcal{L} \leftarrow$ MSE;

while *stopping criterion not satisfied* **do**
  Data collection see Algorithm 2
  $\mathcal{D} \leftarrow$ `DataCollection(`MDP, π_{θ_2}, γ`)`

  Advantage estimation: how much better/worse each action performed compared to the critic's expectation
  for $(s_i, o_i, a_i, g_i) \in \mathcal{D}$ **do**
    $A(s_i, a_i) \leftarrow g_i - \hat{V}_{\theta_1}(s_i, o_i)$;

  state and their returns, from $\mathcal{D}$
  $\mathcal{D}' \leftarrow \big\{ (s, g) \mid (s, o, a, g) \in \mathcal{D} \big\}$

  Critic update with SGD see Algorithm 1
  $\theta_1 \leftarrow$ `SGD(`$\mathcal{D}', \hat{V}, \mathcal{L}, \eta, \theta_1$`)`

  Actor update: increase the probability of actions with positive advantage, decrease it for those with negative advantage
  $$\theta_2 \leftarrow \theta_2 + \eta \sum_{(s_i, o_i, a_i, g_i) \in \mathcal{D}} A(s_i, a_i) \, \nabla_{\theta_2} \log \pi_{\theta_2}(a_i \mid o_i);$$

Return π_{θ_2}

Chapter 4

Explainability in Reinforcement Learning

Today, the majority of Artificial Intelligence (AI) agents with which humans interact are deployed in controlled or low-risk environments. For example, agents capable of playing games such as Go operate in settings where potential errors remain limited in terms of consequences [Silver et al., 2017a]. Even more autonomous systems, such as agents based on Large Language Model (LLM), remain constrained by human supervision mechanisms and are often restricted to tasks such as code generation or information retrieval [Chen et al., 2021, Lewis et al., 2021]. Safety-critical applications integrating machine learning components are still rare. Introducing autonomous agents in such environments, requires the establishment of strong guarantees [Seshia et al., 2022]. These requirements include in particular:

- validating the system's functioning before deployment,
- identifying the origin of a malfunction in case of failure,
- and certifying that an observed failure will not occur again.

The main challenge lies in the fact that the performing agents currently available rely on both Reinforcement Learning (RL) and Deep Learning (DL) [Mnih et al., 2013]. As a result, the mechanisms underlying their decisions cannot be understood directly from the learned model. This lack of explainability makes their use difficult in safety-critical contexts. To address this limitation, a research field has emerged: eXplainable Reinforcement Learning (XRL), a subfield of eXplainable Artificial Intelligence (XAI). The objective of XRL is to develop explainability methods that make it possible to obtain explanations of RL agents. Due to the relevance of the topic and to foster broader real-world adoption of RL, a growing number of research teams are working on developing new explainability methods. This enthusiasm is reflected in a rapid increase in the number of publications, as illustrated in Figure 4.1.[1]

This chapter is organized as follows:

[1]The complete search protocol, including the databases consulted and the full set of query terms, has been archived on Zenodo `https://zenodo.org/records/19387785`.

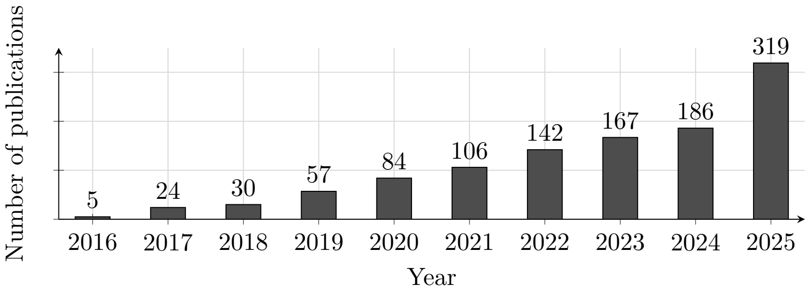


Figure 4.1: Yearly growth of Explainable Reinforcement Learning research with at least one citation on arXiv.

- Section 4.1 begins by clarifying what we mean by an explanation in the context of XAI.
- Section **??** then introduces a taxonomy for categorizing existing XRL methods.
- Section 4.3 provides a comprehensive state-of-the-art review structured around this taxonomy.
- Section **??** concludes the chapter with a summary and discussion.

4.1 Explanation in Artificial Intelligence

In this section, we define what an explanation is in the context of XAI and how it can be obtained. We first introduce the concept of explanation in general. We then describe how this concept applies to XAI. Finally, we further detail the properties that characterize an explainability method in the context of XAI.

4.1.1 Explanation

Establishing what is meant by an explanation is challenging due to the fundamentally multidisciplinary nature of the concept. When we refer to explanation, we are inherently referring to an explanation provided to a human. As soon as humans are involved, the problem is no longer purely computational or mathematical. It involves several fields of research, such as philosophy, psychology, and sociology. So far, these disciplines have not converged on a unified definition of explanation that can be directly applied to XAI [Miller, 2019, Bellucci et al., 2021].

For this reason, we propose the following definition of explanation. An explanation involves two entities: the object (Definition 4.1.1) being explained and the receiver of the explanation. The receiver is a human who aims to understand how the object operates and will be referred to as the observer throughout this manuscript (Definition 4.1.2). The observer seeks to understand the object and, in doing so, builds a mental model of its functioning. An explanation is therefore what allows the observer to refine and correct their mental model of how the object operates (Definition 4.1.4). This refinement is obtained through a transformation process that produces an explanation from the object being explained. We refer to this process as an explainability method (Definition 4.1.5), detailed further in Section 4.1.3. For an observer to understand an explanation, the information it contains must be interpretable (Definition 4.1.3). Information is considered interpretable when it can be directly understood by a human without requiring any additional transformation process.

Definition 4.1.1: Object

The object of an explanation is the system the observer wants to understand.

Definition 4.1.2: Observer

The observer is the human who aims to understand an object. The observer possesses a mental model of the object being explained. This mental model is an internal representation of how the observer believes the system operates. This representation may be incomplete or inaccurate.

Definition 4.1.3: Interpretable

Information is interpretable when it can be directly understood by an observer without requiring any additional transformation process.

Definition 4.1.4: Explanation

An explanation is any interpretable information that enables an observer to refine or correct their mental model of the object of the explanation.

Definition 4.1.5: Explainability Method

An explainability method is a transformation process that produces an explanation from the object being explained.

This definition of explanation provides the foundation for the approach to explainability adopted throughout this manuscript. This work adopts the perspective of *Mechanistic Hypothesis* (Hyp. 3) formalized in Hypothesis 1 [Siegel and Craver, 2024, Zednik, 2019]. Within this framework, explainability is not regarded as a binary property but as a continuum that evolves with the development of scientific knowledge.

Accordingly, the opacity of a system should be interpreted as an epistemic limitation reflecting the current state of knowledge rather than as an intrinsic property of the system itself. This view is consistent with the methodological commitments of scientific inquiry. Scientific progress is driven by the continuous refinement of theories and analytical methods that provide progressively deeper mechanistic accounts of observed phenomena. Consequently, explainability should be regarded as a matter of degree, increasing as these mechanisms become better understood.

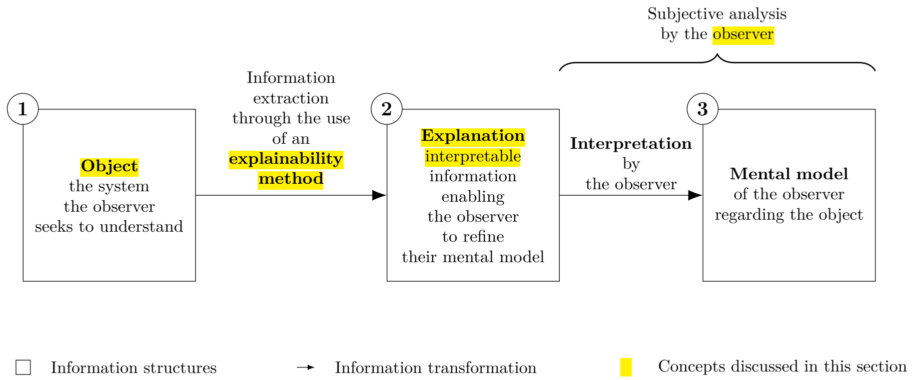


Figure 4.2: Explanation process.

Hypothesis 3: Mechanistic Hypothesis

Every system results from a sequence of causal computations. Because the rules governing these interactions are mechanical, the behavior of the system can be explained. If a system cannot be explained, this reflects a limitation of knowledge rather than an intrinsic property of the system.

This quantitative perspective allows us to express an intuitive idea: not all explanations provide the same level of value [Doshi-Velez and Kim, 2017]. The effectiveness of an explanation depends on its ability to improve the observer's understanding of the system. For example, describing a marginal aspect of a system is different from correcting an observer's misunderstanding of a key mechanism.

Figure 4.2 summarizes this whole explanation process, from the object of the explanation to the mental model refined by the observer, as a chain of three information structures, depicted as boxes and connected by arrows denoting a transformation of information:

(1) The **object** of the explanation, that is, the system the observer seeks to understand.

(2) The **explanation**, the interpretable information produced from the object. This transformation is performed through the use of an explainability method.

(3) The **mental model** held by the observer regarding the object.

A brace spanning boxes (2) and (3) indicates that this second half of the process, from the explanation to the resulting mental model, constitutes a subjective analysis carried out by the observer.

4.1.2 Explainable Artificial Intelligence

Now that the general framework of explanation has been established, we can examine how this concept applies in the context of AI. In the context of XAI, the object of explanation is an AI system. More precisely, XAI emerged as a response to the success of DL models [Gunning and Aha, 2019, Barredo Arrieta et al., 2020]. In these architectures, information is represented in a distributed manner across the network through patterns of activation and connections between computational units. Unlike symbolic approaches, individual components of Neural Networks (NNs) cannot be directly associated with human concepts. From this starting point, we identify two types of approaches:

- **Substitutive explainability:** In this approach, the underlying assumption is that there is nothing to explain within a NN. There is no semantic meaning associated with each computational component, and therefore nothing to interpret. This corresponds to the "black box" view of NNs. Under this assumption, XAI consists of replacing as much of the DL system as possible with alternative AI methods [Rudin, 2019].

- **Intrinsic explainability:** In this approach, it is argued that NNs inherently contain mechanisms that can be explained. The limitation is considered to lie not in the networks themselves, but in the absence of adequate tools to analyze their functioning [Olah et al., 2020].

This manuscript considers methods from both approaches in order to provide a comprehensive overview of the XAI field. According to the *Mechanistic Hypothesis* (Hyp. 3) adopted in this manuscript, we consider that any system can be explained. As a consequence, the perspective adopted throughout this manuscript is that of intrinsic explainability.

4.1.3 Explainability Method in XAI

Now that the general framework of XAI has been established, we need to further detail the properties of an explanation in this context. In our framework, an explanation in the field of XAI can be characterized by three components:

- **Source:** refers to the origin of the information used by the explainability method. The explainability method transforms the raw information produced by the AI system into an explanation that can be understood by an observer.

- **Form:** refers to the representation through which the explanation is communicated. The form defines how the information is presented to the observer and must allow human interpretation. An explanation can take many different forms, including text, images, computer code, and mathematical equations.

- **Subject:** refers to the aspect of the AI system that is being explained. In XAI, this distinction is commonly associated with the difference between local and global explanations. A local explanation describes why an AI system produced a specific output in a given situation. A global explanation aims to describe the general functioning of the AI system.

These three properties of an explanation are entirely determined by the explainability method (Definition 4.1.5) used to produce it. In this context, an explainability method is a transformation process that maps a source of information into an explanation characterized by a specific form and subject.

Research in XAI focuses on developing new methods capable of performing this transformation in order to overcome the lack of interpretability associated with DL models.

4.2 Taxonomy of XRL Methods

Now that the concept of XAI has been defined, we turn to its application in RL, known as XRL. As in XAI, explanations in XRL are produced by explainability methods and can be described in terms of their subject, source, and form. This section reviews the possible variations of these three aspects in the specific context of RL.

4.2.1 Subject

The first question to address is the subject of explanation in the context of RL. To answer this question, it is necessary to identify the different components involved in a RL system and determine which of them constitutes the object of explanation. A RL system can be described through two main components: the environment and the policy.

The environment defines the dynamics in which the agent evolves, including the possible states, observations, actions, and transitions. Environments are simulations designed by humans, and their dynamics are explicitly defined. Therefore, it is assumed that the environment is known to the observer and does not require explanation.

The main object of interest is therefore the agent's policy. The policy represents the decision-making mechanism of the agent. It is a function that maps observations to actions and is learned through interaction with the environment in order to maximize the expected cumulative reward. The policy is the component that encodes the learning process and that XRL aims to explain.

Explaining a policy introduces additional challenges compared with classical XAI settings. In traditional XAI problems, the objective is to explain an isolated decision, such as determining which features of an image led to a classification result. In contrast, RL introduces a temporal dimension into the decision-making process. An action is not only determined by the current observation but also influences future states and rewards. Therefore, understanding the behavior of an agent may require considering a sequence of interactions between the agent and its environment.

This temporal dimension leads to a distinction between two objectives of explanation in XRL:

- **Decision explanations:** focus on identifying the elements of the current observation that influenced the action selected by the policy. This type of explanation is close to classical XAI approaches, as it analyzes the relationship between inputs and outputs while abstracting away the sequential nature of the decision process.

- **Strategy explanations:** aim to characterize the agent's behavior over time. They focus on sequences of decisions and interactions with the environment in order to understand how the agent achieves its long-term objective. This type of explanation is specific to RL, as it addresses the temporal and goal-oriented nature of the learning process.

These two objectives should not be considered as strictly separated categories, but rather as two extremes of a continuum. Understanding the elements of an observation that influence a specific decision can help the observer generalize this information to infer broader aspects of the agent's behavior. Conversely, understanding the agent's overall strategy can provide insights into the reasoning behind individual decisions.

4.2.2 Source

We identify three sources of explanation in RL, all obtained during or after the learning process and containing information about the agent:

- **Policies:** policies are functions that generate a distribution over the action space for a given observation. The objective of RL is to learn a policy that maximizes the expected cumulative future rewards. During the learning phase, the policy is iteratively improved through interactions with the environment.
- **Trajectories:** trajectories represent sequences of successive interactions between the agent and the environment. At each step, the agent selects an action based on its observation, which modifies the environment and produces a reward as well as a new observation. This process continues until the end of the trajectory.
- **Value functions:** value functions provide predictions about the future outcome of a trajectory from a given observation. The most commonly used value function is the critic function, which estimates the expected sum of future rewards. This prediction guides the learning process, as the agent updates its policy to maximize the estimated future return. However, other types of value functions can also be used as sources of explainability.

4.2.3 Form

We identify six forms of explanation in the RL setting:

- **Policy Model:** an interpretable model of the agent's policy function.
- **Saliency Map:** a weighting of the input features representing their influence on the output of the policy function.
- **Counterfactual:** a modified observation obtained by perturbing the original observation in order to produce a different action from the agent.
- **Agent Trajectory:** a sequence of successive interactions between the agent and the environment.
- **Trajectory Prediction:** a prediction about the future evolution of the trajectory produced by a value function.
- **Interaction Model:** an interpretable model of the interactions between the agent and the environment.

Explanation Subject	**Explanation Form**	**Explainability Method Family**	**Source of Explanation**		
			Policy	Trajectories	Value functions
Decision	Policy Model	*Interpretable policy learning*	✓		
		Policy approximation via interpretable model	✓		
	Saliency Map	*Observation perturbation*	✓		
		Observation gradient analysis	✓		
		Attention-based architecture	✓		
	Counterfactual	*Latent space usage for counterfactual generation*	✓		
Strategy	Agent Trajectory	*Hierarchical decomposition*	✓	✓	
		Extraction via critic analysis		✓	✓
	Trajectory Prediction	*Critic enrichment*			✓
	Interaction Model	*Bayesian network learning*		✓	
		Latent space usage for MDP generation	✓		

Table 4.1: Taxonomy of explainability methods in reinforcement learning.

Among the three components characterizing an explanation, the form is the one that varies most significantly across explainability methods. While subjects and sources tend to be relatively stable in the XRL literature, the diversity in how explanations are presented is considerably greater. The list above is by no means definitive and may evolve as new approaches emerge. For this reason, the following section is organized according to the form of explanations and presents existing explainability methods based on this criterion.

4.3 State-of-the-Art Review of XRL Methods

The purpose of this section is to detail the functioning of existing explainability methods (see Table 4.1). They are presented according to the form of explanation they produce, following the taxonomy introduced in Section 4.2.

Each subsection introduces one of the six forms of explanation identified in Section 4.2.3. Within each subsection, each subsubsection is dedicated to a family of methods capable of producing that form of explanation.

A family of methods is defined as a set of methods that share the same source, form, and subject of explanation, as well as a similar procedure for generating explanations. Methods belonging to the same family may differ in their implementation or in the details of the generation process, while relying on the same underlying principle.

4.3.1 Policy Model

The policy function can be represented by a wide variety of function classes. In modern RL, NNs is the most commonly used models for representing policies. Although these models achieve the highest performance, their connectionist nature makes their internal representations non-interpretable.

For this reason, a substitutive approach to XRL aims to replace these models with alternative models considered interpretable. Such models can be understood through direct analysis of their representations. The main classes of interpretable models used to represent policy functions include decision trees [Topin et al., 2021], algorithms [Trivedi et al., 2022], formal logic [Payani and Fekri, 2019], and linear models [Garreau and von Luxburg, 2020].

Explanations derived from an interpretable model primarily provide insights into the decision-making process, since what is modeled is the mapping between an observation and an action. If the model is sufficiently simple or well structured, the observer can form a mental representation of a sequence of decisions across the environment and thus obtain information about the agent's strategy.

The main limitation of these methods is that interpretable models rely on symbolic representations. When the policy to be represented is too complex, achieving comparable performance requires a large number of rules or conditions. The resulting model can quickly become difficult to analyze and lose its interpretability.

For example, representing a policy capable of playing Go using a symbolic model would require considering a very large number of possible situations and specific cases. Even if such a representation were possible, the resulting model would likely become intractable for human analysis due to its complexity. This illustrates why NN-based approaches have become dominant in such domains, as they can learn complex decision strategies without requiring an explicit enumeration of rules.

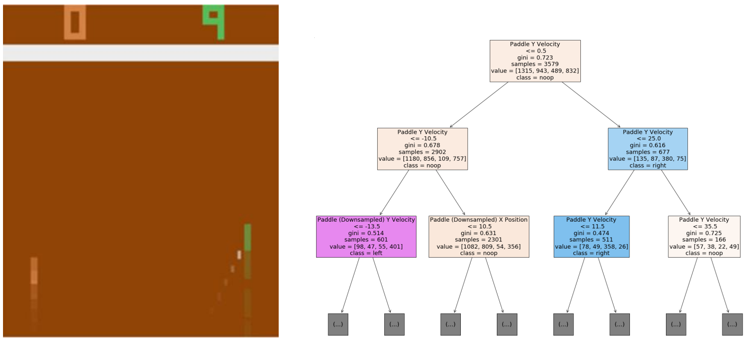
To obtain an interpretable policy model, two families of explainability methods are available: *interpretable policy learning* and *policy approximation via interpretable model*.

Interpretable Policy Learning

This family of methods aims at learning a policy represented by an **interpretable** model [Topin et al., 2021, Trivedi et al., 2022]. It is the most direct approach to addressing the challenges posed by XRL. This type of training requires modifying the Markov Decision Processes (MDP) in order to adapt it to the search space and to the type of policy representation. Moreover, this search space often needs to be reduced to make learning feasible. Such a **reduction** requires the integration of **expert knowledge** into the learning process. With this family of methods, the *interpretable policy function* itself is used to provide explanations to the observer.

Policy Approximation via Interpretable Model

An interpretable model can be trained to approximate a policy learned through deep RL [Sieusahai and Guzdial, 2021]. In this setting, the problem is formulated as a supervised learning task, where the original deep RL policy is used to label each observation with the action it selects (see Figure 4.3a). Using this dataset, the interpretable model is trained to reproduce the behavior of the original policy, yielding for instance the decision tree illustrated in Figure 4.3b. The resulting interpretable model is then used as an explanation for the observer. If this approximation is sufficiently faithful, it can also be deployed in the environment



(a) (b)

Figure 4.3: Policy approximation via an interpretable model [Sieusahai and Guzdial, 2021]. (a) shows an observation from the Atari game “Pong”, and (b) shows the decision tree obtained by approximating a deep reinforcement learning policy trained on this environment.

in place of the original policy, thereby providing a high degree of control over the agent.

4.3.2 Saliency Map

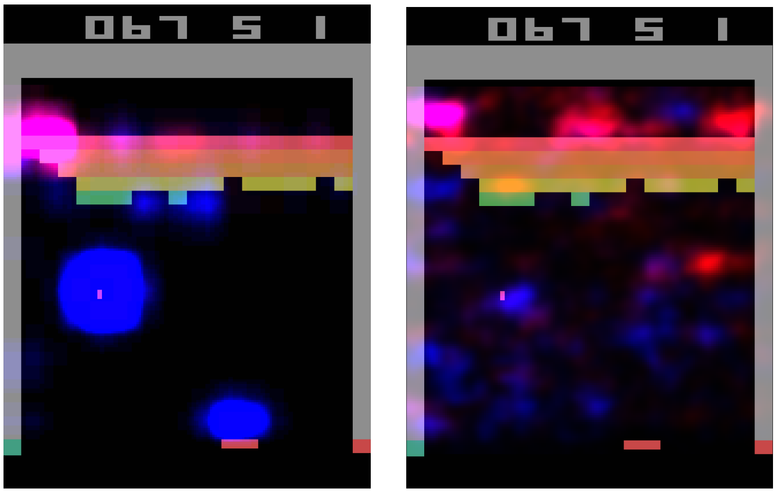
The influence of each part of the input on the action selected by the policy can be visualized using heatmaps (Figure 4.4). In the RL setting, the input corresponds to the observation received by the agent at a given time step. Heatmaps therefore highlight the regions of the observation that have the greatest influence on the action selection process. By providing this information, they allow the observer to identify which elements of the environment are considered important by the policy when making a decision.

A limitation of heatmap-based methods is that the resulting explanations are not always directly meaningful to the observer. As illustrated in Figure 4.4b, interpreting which regions are important often requires human reasoning and involves a degree of subjectivity. This limitation becomes particularly relevant in RL, where the objective is not only to explain a single decision but also to understand the factors influencing a sequence of decisions over time.

Three families of explainability methods can be used to obtain this type of explanation: *observation perturbation*, *observation gradient analysis*, and *attention-based architectures*.

Observation Perturbation

Perturbation analysis consists in applying various **modifications** to the observation in order to study their **impact** on the output of the **policy function** [Greydanus et al., 2018]. The greater the variation in the output caused by a perturbation applied to a given component of the input vector, the higher the **weight** assigned to that component. The magnitude of the perturbation is measured by computing the distance between the original output and the output obtained from the perturbed input. Using this set of weights, a *heatmap* is



(a) (b)

Figure 4.4: Heatmaps obtained by [Greydanus et al., 2018] for the same observation of the Atari game "Breakout". Highlighted areas correspond to the most influential regions for the policy function's decision. (a) is obtained via observation perturbation (see Section 4.3.2), whereas (b) is obtained via observation gradient analysis (see Section 4.3.2).

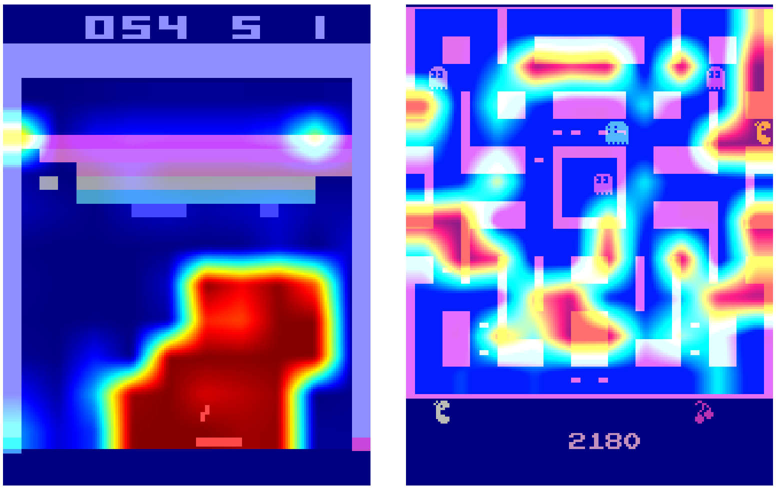
constructed, enabling the observer to visualize which parts of the observation vector are most sensitive in the action-selection process (see Figure 4.4a).

Observation Gradient Analysis

Observation **gradient** analysis refers to a set of methods that require a differentiable policy function in order to extract information from it [Simonyan et al., 2014, Weitkamp et al., 2019, Huber et al., 2019]. To this end, the gradient of the policy function is computed with respect to all **components of the input vector**. For each input component, the resulting gradient provides a weighting that represents its importance in the output of the policy function. Based on these weightings, a *heatmap* is constructed to allow the observer to visually identify the parts of the input vector that are the most influential in the policy function's output (see Figure 4.4b).

Attention-Based Architecture

Attention blocks constitute a NN architecture that makes it possible to weight the relative importance of one part of a vector with respect to the other parts of the same vector [Vaswani et al., 2023]. These weights are computed as a function of the vector itself and are learned by the NN during training. For a given observation, each component of the observation vector is assigned a scalar value. It can therefore be assumed that the components associated with the highest weights are the most influential in determining the output [Mott et al., 2019, Itaya et al., 2021]. This attention-based *heatmap* is then provided to the observer as the explanation (see Figure 4.5a). However, as



(a) (b)

Figure 4.5: Attention-based heatmaps obtained by [Mott et al., 2019] for the Atari games "Breakout" and "Ms. Pac-Man" (see Section 4.3.2). In (a), the highlighted regions are easily interpretable, whereas in (b), the attention is spread across many regions of the observation, making the explanation much harder to interpret.

shown in Figure 4.5b, the attention weights are not always concentrated on a small, easily interpretable set of regions, which can make this type of explanation difficult to exploit.

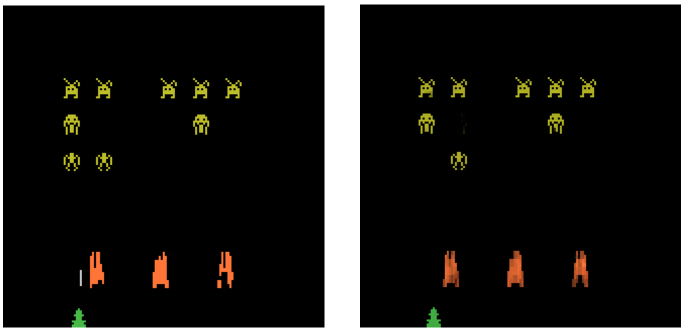
4.3.3 Counterfactual

In the context of RL, counterfactual explanations are used to generate new observations. This new observation, distinct from the original one, leads the agent to adopt an alternative behavior. A counterfactual explanation aims to explain the agent's decision-making process. Modifying the observation informs the observer about the adjustments required for the agent to adopt a different behavior. The family of methods available to generate counterfactuals is the *use of the Latent Space (LS) for counterfactual generation*.

Latent Space Usage for Counterfactual Generation

In this family of methods, **generative models** are used [Bond-Taylor et al., 2021] to exploit the Latent Representation (LR). This type of model learns to generate new data by imitating the training data distribution. In this context, generative models aim to produce alternative observations that are likely to alter the agent's actions (see Figure 4.6) [Olson et al., 2021].

The main challenge of this approach lies in the fact that, to serve as explanations, counterfactuals must be both **close** and **feasible**. A close counterfactual refers to one that does not differ significantly from the original observation. A feasible counterfactual corresponds to an observation that is meaningful within the considered environment.



(a) (b)

Figure 4.6: Counterfactual explanation of the agent's behavior for the game *Space Invaders* [Olson et al., 2021]. (a) is the original observation, for which the agent selects the action "LEFT", whereas (b) is the counterfactual observation, for which the agent selects the action "RIGHT_FIRE".

4.3.4 Agent Trajectory

Until now, we have presented methods that focus on explaining individual decisions made by the agent. We now consider methods that aim to provide insight into the agent's strategy by analyzing its behavior over time.

The most direct approach consists in visualizing agent trajectories, most commonly through complete episodes. By analyzing these trajectories, the observer can infer the dynamics underlying the agent's behavior and how its decisions evolve through time. This type of visualization is not specific to XRL. In practice, visualizing agent episodes is a standard step in the development and evaluation of RL. It provides a simple way to verify that the agent behaves as expected and to detect potential implementation errors.

This simple approach can be further enhanced using families of methods such as *hierarchical decomposition* and *extraction via critic analysis.*

Extraction via Critic Analysis

In this family of methods, the predictions of the critic are used to determine which **trajectories to present to the observer** [Sequeira and Gervasio, 2020]. A set of states is selected from multiple trajectories. For a given state, all possible actions are examined. The state is assigned a value equal to the difference between the critic's prediction for the lowest-valued action and the prediction for the highest-valued action. Trajectories containing the states with the highest values according to this procedure are then chosen to be presented to the observer. These trajectories are considered **critical moments** of the agent and are therefore relevant for analysis. For this family of methods, the *selected trajectories* represent the interpretable information provided to the observer.

Hierarchical Decomposition

Hierarchical agents are composed of a set of **sub-policies** specialized for specific tasks within subsets of the environment's states. This family of methods distributes the optimization of the reward function across multiple models,

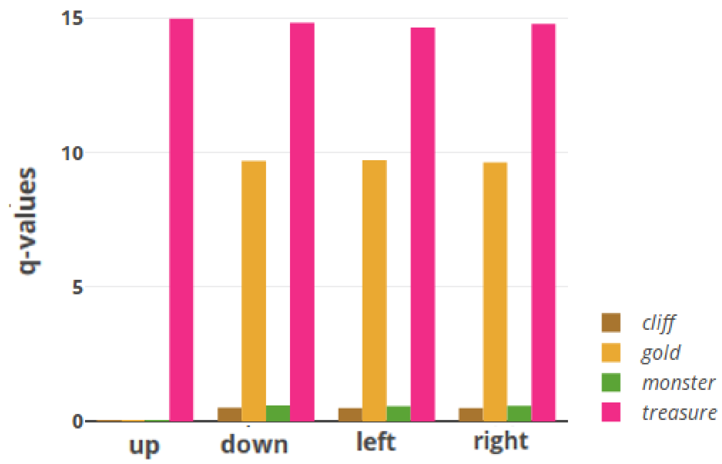


Figure 4.7: Prediction of sub-rewards as a function of the action selected by the agent for a given state [Juozapaitis et al., 2019]. Depending on the action chosen by the agent, one can determine which sub-reward it aims to maximize.

significantly simplifying the learning process. Moreover, this **decomposition** of the policy function can also provide explainability in the agent's strategy [Beyret et al., 2019]. If a sub-policy is selected, it indicates that the agent will execute a specialized behavior. This behavioral specialization provides insight into the strategy the agent employs to maximize the reward function. The agent's *trajectory*, together with the *sub-policy* selected for each action, is used as interpretable information.

4.3.5 Trajectory Prediction

Making predictions about the future of a trajectory allows one to form a representation of what the system expects, based on its training experience. These predictions are produced using value functions. Such functions provide insights into the future evolution of the agent's trajectory. By their nature, this form of explanation informs the observer about the strategy adopted by the agent. Value functions provide an estimate of the expected cumulative reward from a given state. Consequently, a value function maps an observation to a scalar value in $\mathbb{R}$.

This aggregated prediction provides limited information about the underlying factors that contribute to this value. It does not directly reveal which aspects of the task are being optimized or how the predicted return is composed. To improve the interpretability of value-based explanations, a family of methods known as *critic enrichment* can be employed.

Critic Enrichment

In order to obtain interpretable information from the predictions of the **critic**, it is possible to **decompose** the critic into multiple **sub-functions** [Juozapaitis et al., 2019]. The reward function is thus decomposed into multiple sub-reward functions, each representing a **sub-objective** of the task to be accomplished. Similarly,

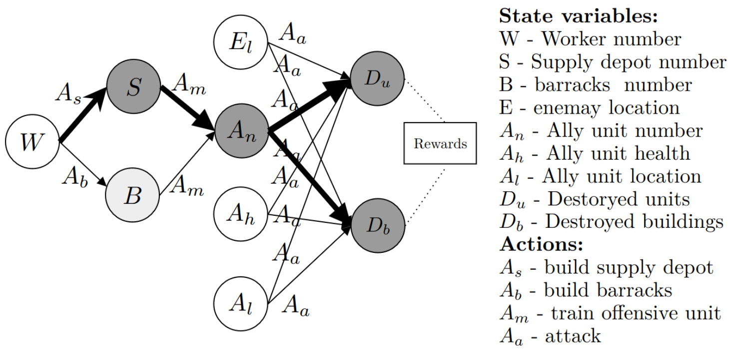


Figure 4.8: Agent–environment interaction model for the video game *StarCraft II* [Madumal et al., 2020]. Nodes represent random variables, and edges represent causal relationships between them.

for each of these sub-reward functions, corresponding sub-critic functions are defined, along with a function that combines their outputs. During training, the agent selects its actions so as to maximize this combination function of the sub-critics. Using this approach, for each observation and action, a prediction is obtained regarding the sub-objectives the agent is attempting to maximize (see Figure 4.7). This *prediction* is then used as the explanation for the observer.

4.3.6 Interaction Model

Modeling the interactions between an agent and its environment makes it possible to understand how the agent's decisions are made and what impact they have on the environment. By incorporating these interactions into an explanatory model, one can better grasp the complex relationships between the agent and its environment, thereby identifying the determining factors in the decision-making process. By explicitly handling the agent–environment interaction, this type of explanation aims to explain the agent's strategy. The families of methods used to obtain explanations in the form of agent–environment interactions are: *Bayesian network learning* and *LS usage for MDP generation*.

Bayesian Network Learning

The evolution of an agent within an environment can be observed as a set of **random variables** connected through a **Bayesian network** [Madumal et al., 2020]. Random variables are defined to represent both external aspects (the environment) and internal aspects of the agent (the actions). By analyzing trajectories, it becomes possible to infer causal relationships among these random variables. At the end of this process, a weighted causal graph is obtained, which serves as an explanation for the observer (see Figure 4.8). This *Bayesian network*, representing the interactions between the agent and the environment, is then used as the explanation.

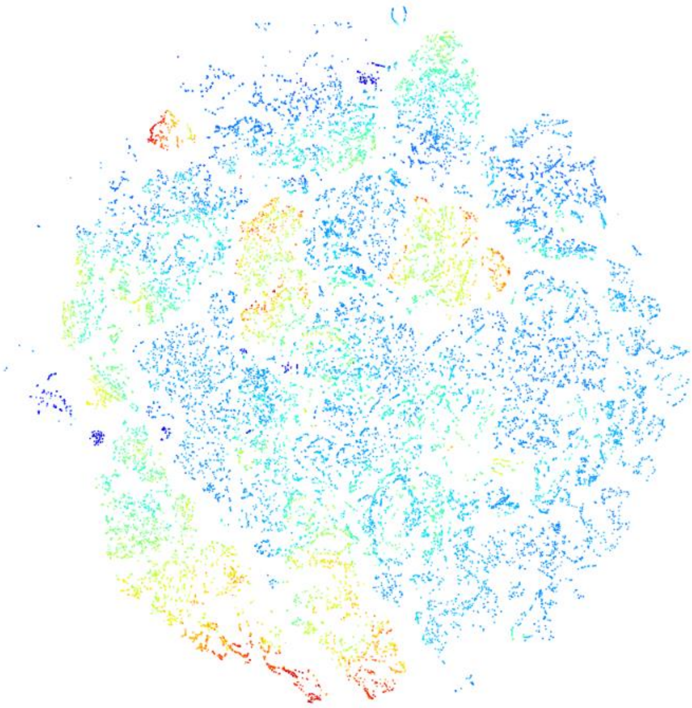


Figure 4.9: *t-SNE* projection of a neural network's latent space [Zahavy et al., 2017]. Each point corresponds to a state, and its color represents the value function's prediction for that state, with warmer colors indicating higher predicted values.

Latent Space Usage for MDP Generation

Each layer of a NN can be viewed as a projection from one space to another [Zahavy et al., 2017]. As information propagates through successive layers of a NN, it becomes increasingly abstract. It can therefore be assumed that two vectors that are close in this projected space are also close in terms of the agent's perception. Based on this observation, it is possible to **redefine an MDP** by leveraging the **abstract representations** produced by the final layers of a NN. Once the new MDP has been constructed, a model of the interaction between the agent and its environment is obtained. This model, represented as a *graph*, is then used as the explanation.

Figure 4.9 illustrates this idea using a two-dimensional *t-SNE* projection of the LS of the NN used to represent the policy. Each point is colored according to the value function's prediction for the corresponding state. An agreement can be observed between the structure of this projected space and the critic's predictions: states that are assigned similar values by the critic also tend to be located close to one another in the projection. This agreement makes it possible to manually identify clusters of states, which likely correspond to states that are perceived as similar by the agent.

4.4 Conclusion

The taxonomy and the state-of-the-art review presented in this chapter make it possible to draw three main observations.

4.4.1 Absence of Explanation of the Learning Process

Regarding the subject of explanation, Table 4.1 shows that none of the reviewed methods addresses the learning process itself. Such an explanation would concern how the policy is progressively updated throughout training, and why it evolves over successive iterations to converge toward a particular solution. Several reasons explain this absence.

Among the different machine learning paradigms, RL is arguably the one that would benefit the most from explanations of the learning process. Unlike supervised or unsupervised learning, it relies on an explicit exploration–exploitation trade-off, making the learning trajectory itself an informative object of explanation. Most XRL methods, however, are inherited from the XAI literature, where explanations focus almost exclusively on the behavior of already-trained models. This limitation has naturally carried over to XRL.

From a practical perspective, explaining the learning process is also of limited industrial interest. In most real-world applications, practitioners deploy an already-trained agent, while training itself takes place offline. Users are generally interested in understanding the behavior of the final policy rather than the optimization process that produced it.

Explaining the learning process also raises significant methodological challenges. Training in RL can be viewed as the iterative improvement of a policy through repeated interactions with the environment. In this sense, explaining learning would amount to explaining the evolution of an entire sequence of policies rather than a single one. Current XRL methods still struggle to provide comprehensive explanations of an individual policy, so extending them to a full training trajectory appears considerably more difficult. Explanations of the learning process therefore remain an open research direction rather than a straightforward extension of existing XRL methods.

4.4.2 Lack of Unified Metrics for Explainability

The diversity of explanation forms identified in this review also reveals another limitation shared by nearly all families of XRL methods: the absence of reliable metrics for evaluating explainability. In many studies, explanations are not evaluated with human participants. Instead, authors focus on technical properties of the proposed method, such as fidelity, sparsity, or computational efficiency. When human evaluations are conducted, they usually rely on subjective satisfaction questionnaires. Although useful, self-reported satisfaction remains prone to numerous biases and does not necessarily indicate that the explanation has improved the observer's mental model or understanding of the agent.

This limitation is not specific to XRL: it reflects a broader challenge affecting XAI as a whole. In our view, this challenge stems from a historical development largely driven by the computer science community. Explainability has consequently often been studied from an algorithmic or mathematical perspective, with less attention given to how humans actually benefit from explanations.

Our definition of explanation explicitly places the observer at the center of the explanatory process. From this perspective, the value of an explanation should not only depend on its technical properties but also on its ability to improve the observer's understanding. However, the field still lacks objective and standardized metrics capable of measuring this aspect.

Addressing this challenge will require stronger interdisciplinary collaboration between computer science and disciplines such as cognitive science, psychology, philosophy, and sociology. These fields provide the theoretical and experimental tools needed to study human understanding and to develop evaluation protocols that measure the actual effectiveness of explanations. The absence of such collaborations is likely a consequence of the relative youth of XAI as a research field, whose theoretical foundations and evaluation methodologies are still being established.

4.4.3 Toward Intrinsic Explainability in XRL

Our review also suggests that current XRL research remains largely dominated by substitutive explainability approaches (Section 4.1). Most existing methods explain RL agents through surrogate models or external explanations rather than by analyzing the neural network itself. While these approaches often improve interpretability, they inevitably discard part of the information contained in the learned representation.

We argue instead that explainability should progressively move toward intrinsic approaches, which analyze the internal mechanisms of the agent rather than bypassing them. From this perspective, the neural network should not be regarded as an inherently opaque black box, but rather as a complex representation whose apparent opacity mainly reflects the current limitations of our analysis tools (Hypothesis 3). This viewpoint is consistent with the discussion developed in Chapter 2, where we argue that the notion of a black box is only temporary.

Among the methods reviewed, *latent space usage for MDP generation* (Section 4.3.6) appears closest to this intrinsic perspective. Instead of replacing the learned representation, it directly exploits the latent space to identify meaningful concepts that can support the explanation of the agent's behavior. However, this approach still relies on a largely manual and qualitative analysis of the latent space, which limits its reproducibility and its ability to produce systematic explanations. The next chapter builds upon this idea and proposes a mechanistic XRL approach based on latent-space analysis. Rather than constructing a surrogate explanation, it aims to explain the agent by studying its internal representations.

Chapter 5

Clustering Agent Representations

As discussed in the previous chapter, current explainability methods for Reinforcement Learning (RL) still lack intrinsic approaches capable of explaining the internal mechanisms of deep neural policies. To the best of our knowledge, only [Zahavy et al., 2017] have proposed exploiting the Latent Space (LS) learned by neural policies in order to identify the concepts used by the agent. Its main idea is to analyze the Latent Representations (LRs) produced by the Neural Network (NN) and organize them into clusters that correspond to high-level concepts.

While the core idea of this paper is closely aligned with our vision of explainability in RL, this approach suffers from two important limitations. First, the clusters are manually constructed. Such a procedure heavily depends on the user's expertise and neither guarantees reproducibility or scales. Second, no evaluation protocol is proposed to assess the quality of the obtained clusters. Their relevance is essentially determined through visual inspection.

The Clustering Agent Representation (CAR) method proposed in this chapter addresses these two limitations. It automates concept discovery by relying on unsupervised clustering algorithms to identify the Concept Structure (CS) of the LS. It introduces an objective evaluation metric based on the stability of the extracted representations across independently trained policy.

This chapter is organized as follows:

- Section 5.1 lays out the theoretical assumptions underlying the proposed CAR method, which we term the *Representation Convergence Hypothesis* (Hyp. 4).
- Section 5.2 then details the CAR method itself, covering both the extraction of LRs using surrogate policies and the automatic detection of CSs through clustering.
- Section 5.3 finally introduces the metrics we propose for evaluation.

5.1 Representation Convergence Hypothesis

The CAR method relies on a assumption, referred to as the *Representation Convergence Hypothesis* (Hyp. 4). This hypothesis follows the two assumptions established previously throughout this manuscript.

The *Mechanistic Hypothesis* (Hyp. 3) rejects the notion that deep NNs constitute irreducible black boxes. Adopting a mechanistic perspective, we consider that every process results from a sequence of causal computations. Although these computations may be highly complex, they remain explainable through the analysis of the internal mechanisms.

The *Concept Hypothesis* (Hyp. 1) states that NNs achieve generalization by learning concepts. During optimization, the network progressively organizes its LS into representations that capture the information required to solve the task.

Following these hypotheses, it becomes possible to explain the NN's process by understanding the organization of its LS. This family of approaches is commonly referred to as mechanistic interpretability.

In mechanistic interpretability, universality refers to the consistency with which a concept is recovered across different models or independent extraction processes [Templeton et al., 2024]. We develop the *Representation Convergence Hypothesis* (Hyp. 4) provides a theoretical explanation for this phenomenon. Under this assumption, the universality of a concept is not merely an empirical observation but the expected consequence of a common underlying CS shared by independently learned representations.

> **Hypothesis 4: Representation Convergence Hypothesis**
>
> For a given task and a given NN architecture, there exists a privileged CS, which emerges because it constitutes an efficient representation required to solve the task.

It is important to clarify that this hypothesis does not imply convergence at the parameter level. Independently trained models are expected to converge toward different sets of weights. The proposed hypothesis concerns a abstract level of representation. We assume that the conceptual organization induced by their LR converges toward an equivalent CS, even though the numerical implementation of the networks differ.

The remainder of this chapter builds upon this hypothesis. This principle forms the theoretical foundation of the *Clustering Universality* metric, introduced in Subsection 5.3.3.

5.2 Method

Following the work of [Zahavy et al., 2017], we analyze the LS through clustering techniques, placing our approach within the broader field of Deep Clustering. This design choice is motivated by our rejection of the *Superposition Hypothesis* (Hyp. 2) and, more generally, of any view that assumes a linearly organized CS, as discussed in Section 2.5.

The CAR methodology is composed of two main stages. The first stage trains multiple independent NNs, called surrogate policies, which implement the same decision policy. The second stage clusters their LRs in order to identify the underlying conceptual organization.

This section is organized as follows. In the first subsection, we justify the use of multiple surrogate policies in order to study the stability of representations. In the second subsection, we detail how these surrogate policies are trained so as to best match the original policy.

5.2.1 Multiple surrogate policies

The CAR method was developed with the aim of verifying the *Representation Convergence Hypothesis* (Hyp. 4). To test this hypothesis, several NNs must be trained independently and their LRs compared. If similar CSs consistently emerge, this provides empirical evidence supporting the Representation Convergence Hypothesis.

In supervised learning, this protocol is straightforward since several models can be independently trained on the same target function. RL introduces an additional difficulty. Because of stochastic exploration, independently trained agents are not guaranteed to converge toward the same policy. Even if they achieve comparable performance, they may solve the task using different strategies. Making it impossible to determine whether differences observed in the LS originate from the learning dynamics or from genuine differences between the learned policies.

To overcome this limitation, we consider an already trained initial policy. This policy is then imitated using imitation learning [Ross et al., 2011]. To this end, we independently train several surrogate NNs in a supervised manner. Since all surrogate models approximate the same target function, they are converge toward equivalent decision policies. This makes it possible to investigate whether equivalent policies also exhibit equivalent conceptual organizations.

It is important to emphasize that the use of surrogate policies does not make CAR a surrogate explainability method, as defined in Subsection 4.1.2. The surrogate policies remain deep NNs. Their role is not to simplify the decision process but to provide multiple independent realizations of the same policy function, enabling the study of representation convergence.

5.2.2 Training surrogate policies

Throughout the remainder of this manuscript, the initial policy we aim to explain is denoted as π. This initial policy is obtained without imposing any constraints on its origin. It may result from classical or deep reinforcement learning [Sutton and Barto, 2018], alpha-beta algorithm [Knuth and Moore, 1975], or even from demonstrations provided by humans [Ross and Bagnell, 2010]. The crucial requirement is the availability of a dataset containing a sufficient number of projections from the observation space $\mathcal{O}$ to the action space $\mathcal{A}$ induced by π.

The replicated policies are referred to as surrogate policies, denoted by π'. For any observation O, the distribution over the action space produced by π' should approximate that of π as closely as possible.

The surrogate policy π' is implemented as a NN parameterized by Θ. This network is composed of two components: an encoder and a decoder, respectively highlighted by the blue and black dashed rectangles in Figure 5.1. The encoder

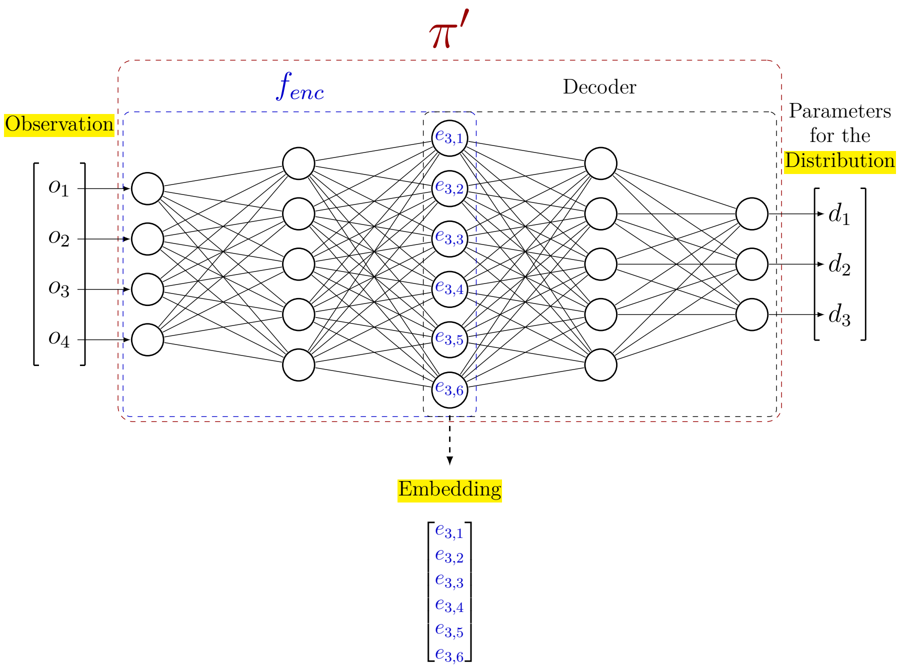


Figure 5.1: Example of a surrogate policy architecture π'.

f_{enc} maps the observation O to the embedding. The decoder, which reconstructs the action distribution parameters from the embedding.

To train π' to accurately replicate the action distribution, we define a global loss function $\mathcal{L}$ optimized via gradient descent as introduced in Section 1.4. This loss, defined in Equation (5.1), is computed as the Mean Squared Error (MSE) between the parameters of the action distributions, obtained by applying the function p, which maps a distribution to its parameterization. In practice, this function does not need to be explicitly defined, as neural policies directly output the parameters of the action distribution. Once training is complete, this loss ensures that π' closely approximates the behavior of the original policy π. Figure 5.2 summarizes this overall procedure.

$$\min_{\Theta} \sum_{O \in \mathcal{O}} \mathcal{L}\left(O\right) = \min_{\Theta} \sum_{O \in \mathcal{O}} \text{MSE}\Big(p\big(\pi\left(O\right)\big), p\big(\pi'_{\Theta}\left(O\right)\big)\Big) \tag{5.1}$$

We denote by $\mathcal{E}$ the set of embeddings obtained by applying f_{enc} to a subset of observations $\mathbb{O} \subseteq \mathcal{O}$.

On the set $\mathcal{E}$, we use a clustering function f_{clust} to partition the point cloud of embedded observations into distinct clusters. Each observation is assigned a cluster label based on its proximity in the LS, resulting in the clustering $\mathcal{C}$. Algorithm 5 summarizes the complete CAR procedure.

To demonstrate this clustering faithfully reflects the agent's internal decision-making process, we introduce a set of evaluation metrics, which will be detailed in the following section.

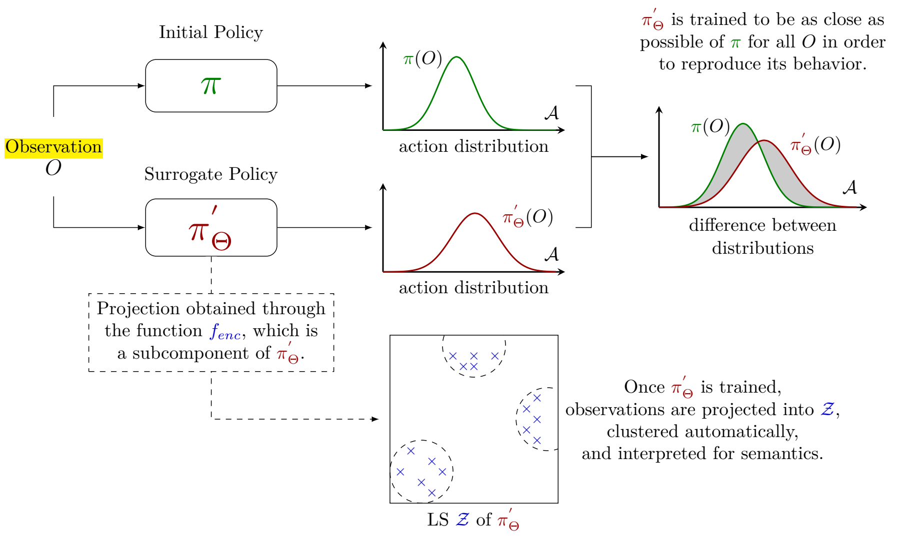


Figure 5.2: Illustration of the Clustering Agent Representation method.

5.3 Evaluation Metrics

A key contribution of this work is the development of three evaluation metrics, designed to assess the effectiveness of CAR:

- ***Behavior Reconstruction*** : it measures how well a surrogate policy faithfully reproduces the behavior of the original policy by comparing their cumulative rewards.
- ***Clustering Universality*** : it evaluates the stability of clusterings by measuring the similarity of clusters obtained from independently trained surrogate policies.
- ***Abstraction Score*** : it quantifies the extent to which LS clusters capture abstractions rather than directly reflecting the structure of observations or actions.

The remainder of this section details the motivation for each metric and describes how they are calculated. In the first subsection, we develop the *Behavior Reconstruction*. We then introduce the Adjusted Rand Index (ARI) measure, which serves as the foundation for the two other metrics introduced in the subsequent subsections, namely *Clustering Universality* and *Abstraction Score*.

The following notations will be adopted for the rest of the manuscript: we consider n surrogate policies, denoted as $\pi'_1, \ldots, \pi'_n$, trained as described in Section 5.2. Each surrogate policy is associated with a projection function $f_{enc_1}, \ldots, f_{enc_n}$, which is applied to the set of observations $\mathbb{O}$, yielding embeddings $\mathcal{E}_1, \ldots, \mathcal{E}_n$. A clustering function is then applied to each embedding set, producing partitions $\mathcal{C}_1, \ldots, \mathcal{C}_n$.

Algorithm 5: Clustering Agent Representation (CAR)

Input:
Original policy π
Observation dataset $\mathbb{O}$
Number of surrogate policies n
Surrogate policy architecture π' (encoder f_{enc}, decoder)
Learning rate η
Clustering function f_{clust}
Output:
Trained surrogate policies $\pi'_{\Theta_1}, \ldots, \pi'_{\Theta_n}$
Clusterings $\mathcal{C}_1, \ldots, \mathcal{C}_n$

```
// Behavioral cloning dataset built from the target policy π
D ← {(O, p(π(O))) | O ∈ 𝕆};
// MSE loss between action-distribution parameters, as defined in Equation (??)
L ← MSE;
// Independently train n surrogate policies by behavioral cloning
for i = 1, ..., n do
    Initialize Θ_i randomly;
    // Behavioral cloning of π via SGD, see Algorithm 1
    Θ_i ← SGD(D, π', L, η, Θ_i);
// Cluster the latent representations of each surrogate policy
for i = 1, ..., n do
    // Embeddings produced by the encoder of π'_Θi
    E_i ← {f_enc_i(O) | O ∈ 𝕆};
    // Partition of the embedded observations into clusters
    C_i ← f_clust(E_i);
Return (π'_Θ1, ..., π'_Θn), (C_1, ..., C_n)
```

5.3.1 Behavior Reconstruction

We argue that explaining a function b that closely approximates another function a is essentially the same as explaining a. If b accurately captures the key decision patterns of a, understanding b provides meaningful insights into how a works [Guidotti et al., 2018]. In our setting, this means that generating explanations for π' that faithfully reproduce π amounts to explaining π directly. To verify that π' is indeed a faithful copy, we introduce the *Behavior Reconstruction* metric.

This metric is computed by comparing the cumulative rewards obtained by the original policy π and the surrogate policies π' over m evaluation episodes. The behavior reconstruction metric is then defined as:

$$\textit{Behavior Reconstruction} = \frac{1}{n \sum_{i=1}^{m} g_{\pi}^{i}} \sum_{j=1}^{n} \sum_{k=1}^{m} g_{\pi'_j}^{k} \tag{5.2}$$

In imitation learning, it is unlikely to expect the surrogate policy π' to outperform the initial policy π it aims to approximate through supervised learning [Ross and Bagnell, 2010]. Consequently this metric ranges in $[0, 1]$. Values approaching 1 indicate minimal loss in reproducing π, while values close to 0 indicate poor reproduction.

We deliberately adopt a reward-based definition rather than alternatives based on actions or action distributions (e.g., training loss, accuracy, KL divergence [Shlens, 2014], or optimal transport [Peyré and Cuturi, 2020]). Since surrogate policies are trained in a supervised manner, they cannot develop strategies beyond those demonstrated by the initial policy. This means that if the rewards achieved are comparable, the surrogate policy is effectively copying the original policy's behavior.

5.3.2 Adjusted Rand Index

Within CAR, concepts are assumed to correspond to clusters in the LS. Combined with the *Representation Convergence Hypothesis* (Hyp. 4), this leads to a direct experimental prediction: if independently trained surrogate policies converge toward the same conceptual organization, then clustering their LRs should produce similar partitions.

When a particular observation is processed by the network, it is expected to activate the same concept regardless of the specific neural implementation of the policy. If independently trained policies converge toward the same conceptual organization, a given observation should activate the same concept across all models. As a consequence, observations grouped together within a cluster for one policy should also be grouped together for another independently trained policy.

In the clustering literature, the most used measures for comparing two partitions is the ARI [Hubert and Arabie, 1985]. This measure quantifies the agreement between cluster assignments independently of the spatial organization of the clusters. Both metrics introduced later in this chapter, namely *Clustering Universality* and *Abstraction Score*, rely on the ARI. Since the ARI is a complex measure, we review its definition and discuss its main properties.

The ARI is derived from the Rand Index (RI) [Rand, 1971], which measures the agreement between two partitions by considering every possible pair of point cloud. For each pair, four situations are possible: the point may belong to the same cluster in both partitions, to different clusters in both partitions, or they may disagree by being grouped together in only one of the two partitions. The RI measures the proportion of pairs for which the two partitions agree:

$$\mathrm{RI} = \frac{\text{number of agreeing pairs}}{\text{total number of pairs}}.$$

Although the RI is intuitive, it suffers from an important limitation. In most clustering problems, the majority of pairs of points belong to different clusters. Consequently, even two completely independent partitions will agree on a large number of pairs, simply because both assign them to different clusters. This effect becomes more pronounced as the number of points or clusters increases.

The ARI addresses this issue by accounting for the agreement that would be expected purely by chance. It compares the observed agreement RI to the expected agreement under random partitions $\mathbb{E}[\mathrm{RI}]$. The score is then normalized by the maximum agreement achievable beyond this random baseline, max(RI) − $\mathbb{E}$[RI]. As a result, an ARI of 0 corresponds to the level of agreement expected from random clusterings, while an ARI of 1 indicates perfect agreement between the two partitions:

$$\mathrm{ARI} = \frac{\mathrm{RI} - \mathbb{E}[\mathrm{RI}]}{\max(\mathrm{RI}) - \mathbb{E}[\mathrm{RI}]}$$

In practice, the ARI is computed directly from the cluster assignments of the two partitions. Let clustering A be composed of partitions $\{A_i\}$ and clustering B of partitions $\{B_j\}$. Let $a_i = |A_i|$ denote the number of samples in partition i of clustering A, and $b_j = |B_j|$ the number of samples in partition j of clustering B. Furthermore, let $n_{ij} = |A_i \cap B_j|$ be the number of samples shared by partition i of clustering A and partition j of clustering B, and let n be the total number of samples. With these definitions, the ARI can be computed as:

$$\mathrm{ARI} = \frac{\sum_{i,j}\binom{n_{ij}}{2} - \left[\sum_i \binom{a_i}{2}\sum_j \binom{b_j}{2}\right]/\binom{n}{2}}{\frac{1}{2}\left[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2}\right] - \left[\sum_i \binom{a_i}{2}\sum_j \binom{b_j}{2}\right]/\binom{n}{2}}$$

5.3.3 Clustering Universality

Having introduced the ARI, we can now define the *Clustering Universality* metric. While the ARI quantifies the similarity between two conceptual partitions, the objective of CAR is to assess the consistency of conceptual organizations across an entire set of independently trained surrogate policies.

To this end, the same set of observations is projected into the LS of each surrogate policy and clustered. Under the Representation Convergence Hypothesis, these independently obtained partitions are expected to be similar. *Clustering Universality* quantifies this by computing the mean pairwise ARI:

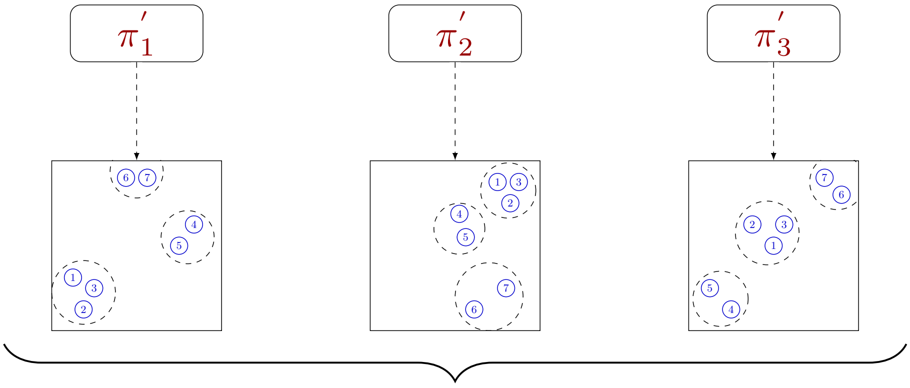
$$\textit{Clustering Universality} = \frac{1}{\binom{n}{2}} \sum_{i=1}^{n} \sum_{j=0}^{i-1} \mathrm{ARI}(\mathcal{C}i, \mathcal{C}j). \tag{5.3}$$

This metric takes values in $[-1, 1]$. A value close to 1 indicates that surrogate policies produce highly similar partitions, suggesting that the clusters reflect intrinsic properties of the policy. Conversely, a value near 0 implies that the similarity between partitions is no better than that of two randomly generated clusterings, meaning that the clustering fails to capture any meaningful structure in the policy. A negative value, on the other hand, signals that the agreement between partitions is worse than what would be expected by chance, indicating a systematic disagreement between the clusterings.

Figure 5.3 illustrates *Clustering Universality* in the ideal case. It shows three LS obtained by projecting the same seven observations, labeled 1 to 7, through three independently trained surrogate policies. Although the embeddings occupy different positions, the clusters group exactly the same elements in all three cases: the partitioning is therefore identical.

5.3.4 Abstraction Score

If *Clustering Universality* is high, it becomes important to determine what underlies this stability. Two possibilities arise:



Clustering Universality $= 1$

Figure 5.3: Illustration of *Clustering Universality* in the ideal case where projections of the same observations are grouped within the same clusters across all surrogate policies.

(i) the LS merely reproduces the structure of the observation space or action distribution, offering no abstraction, as defined in Definition 2.1.1;

(ii) the LS captures similarities in decision-making processes, yielding behaviorally meaningful clusters.

To distinguish among these cases, we introduce the *Abstraction Score*. This metric contextualizes latent clusterings by examining how their patterns align with reference structures derived from the raw observation and action data. We consider : $\mathcal{C}_{obs}$, the clustering in the observation space $\mathbb{O}$, and $\mathcal{C}_{act}$, the clustering in the action-space distribution parameters. The clusterings $\mathcal{C}_{obs}$ and $\mathcal{C}_{act}$ are performed using the same algorithm, distance metric, and hyperparameters as those used for $\mathcal{C}_i$ in the LS, ensuring that any biases introduced by the clustering procedure itself are minimized.

For each latent clustering $\mathcal{C}_i$, we define its similarity to these baselines using the ARI:

$$A_i^{obs} = \mathrm{ARI}(\mathcal{C}_i, \mathcal{C}_{obs})$$
$$A_i^{act} = \mathrm{ARI}(\mathcal{C}_i, \mathcal{C}_{act})$$

The *Abstraction Score* is subsequently calculated as one minus the average of the maximum absolute similarity values across these two reference baselines:

$$\textit{Abstraction Score} = 1 - \tfrac{1}{n}\textstyle\sum_{i=1}^{n} \max\left(\left|A_i^{obs}\right|, \left|A_i^{act}\right|\right) \tag{5.4}$$

The *Abstraction Score* ranges from $[0, 1]$. A score around 0 means that the latent clusters are strongly aligned with either the observation or the action space, reflecting minimal abstraction, cases (i). A score close to 1 indicates that the latent clusters are independent of both the observation and action spaces, capturing abstractions that meaningfully reflect the agent's behavior, case (ii).

The following chapter presents the results of applying these metrics to specific examples.

Acronyms

AI Artificial Intelligence 48, 51, 52
ARI Adjusted Rand Index 70, 72–74

CAR Clustering Agent Representation 66–70, 72, 73
CS Concept Structure 17, 22, 23, 26–28, 30, 31, 66–68

DL Deep Learning 1–4, 6, 7, 9, 13, 14, 35, 48, 52, 53

LLM Large Language Model 2, 14, 19, 20, 26, 27, 36, 48
LR Latent Representation 14–16, 18, 20–31, 59, 66–68, 72
LS Latent Space 13–19, 21–25, 27–30, 59, 62, 63, 66–70, 72–74

MDP Markov Decision Processes 34–39, 42–45, 47, 56, 62, 63
MLP MultiLayer Perceptron 4, 6, 7
MSE Mean Squared Error 4, 30, 45, 47, 69, 71

NN Neural Network 1–12, 14–19, 24, 29, 31, 32, 35, 43, 44, 52, 55, 56, 58, 63, 66–68

PPO Proximal Policy Optimization 35

RI Rand Index 72
RL Reinforcement Learning 14, 33–44, 46, 48, 53–57, 59, 60, 64–66, 68

SAE Sparse AutoEncoders 24, 26, 27, 30
SGD Stochastic Gradient Descent 10

XAI eXplainable Artificial Intelligence 48, 49, 51–53, 64
XRL eXplainable Reinforcement Learning 48, 49, 53, 55, 56, 60, 64, 65

Bibliography

[Barredo Arrieta et al., 2020] Barredo Arrieta, A., Díaz-Rodríguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., García, S., Gil-López, S., Molina, D., Benjamins, R., Chatila, R., and Herrera, F. (2020). Explainable artificial intelligence (xai): Concepts, taxonomies, opportunities and challenges toward responsible ai. Information Fusion, 58:82–115. **Article**.

[Barto et al., 1983] Barto, A. G., Sutton, R. S., and Anderson, C. W. (1983). Neuronlike adaptive elements that can solve difficult learning control problems. IEEE Transactions on Systems, Man, and Cybernetics, 13(5):835–846.

[Bellman, 1957] Bellman, R. E. (1957). Dynamic programming.

[Bellucci et al., 2021] Bellucci, M., Delestre, N., Malandain, N., and Zanni-Merk, C. (2021). Towards a terminology for a fully contextualized xai. Procedia Computer Science, 192:241–250. **Article**.

[Bengio et al., 2014] Bengio, Y., Courville, A., and Vincent, P. (2014). Representation learning: A review and new perspectives.

[Beyret et al., 2019] Beyret, B., Shafti, A., and Faisal, A. A. (2019). Dot-to-dot: Explainable hierarchical reinforcement learning for robotic manipulation. In 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE. **Article**.

[Bond-Taylor et al., 2021] Bond-Taylor, S., Leach, A., Long, Y., and Willcocks, C. G. (2021). Deep generative modelling: A comparative review of vaes, gans, normalizing flows, energy-based and autoregressive models. **Article**.

[Bricken et al., 2023] Bricken, T., Templeton, A., Batson, J., Chen, B., Jermyn, A., Conerly, T., et al. (2023). Towards monosemanticity: Decomposing language models with dictionary learning. Transformer Circuits Thread. **Web**.

[Brown et al., 2020] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. Advances in neural information processing systems, 33:1877–1901.

[Burda et al., 2018] Burda, Y., Edwards, H., Pathak, D., Storkey, A., Darrell, T., and Efros, A. A. (2018). Large-scale study of curiosity-driven learning.

[Chen et al., 2021] Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., et al. (2021). Evaluating large language models trained on code. **Article**.

[Coates et al., 2011] Coates, A., Ng, A. Y., and Lee, H. (2011). An analysis of single-layer networks in unsupervised feature learning. In Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS), pages 215–223.

[Covington et al., 2016] Covington, P., Adams, J., and Sargin, E. (2016). Deep neural networks for youtube recommendations. In Proceedings of the 10th ACM conference on recommender systems, pages 191–198.

[Doshi-Velez and Kim, 2017] Doshi-Velez, F. and Kim, B. (2017). Towards a rigorous science of interpretable machine learning. **Article**.

[Elhage et al., 2022] Elhage, N., Hume, T., Olsson, C., Schiefer, N., Henighan, T., Kravec, S., et al. (2022). Toy models of superposition. Transformer Circuits Thread. **Web**.

[Garreau and von Luxburg, 2020] Garreau, D. and von Luxburg, U. (2020). Explaining the explainer: A first theoretical analysis of lime. In Chiappa, S. and Calandra, R., editors, Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics, volume 108 of Proceedings of Machine Learning Research, pages 1287–1296. PMLR. **Article**.

[Goodfellow et al., 2016] Goodfellow, I., Bengio, Y., Courville, A., and Bengio, Y. (2016). Deep learning, volume 1. MIT press Cambridge.

[Greydanus et al., 2018] Greydanus, S., Koul, A., Dodge, J., and Fern, A. (2018). Visualizing and understanding atari agents. **Article**. **Code**.

[Guidotti et al., 2018] Guidotti, R., Monreale, A., Ruggieri, S., Turini, F., Pedreschi, D., and Giannotti, F. (2018). A survey of methods for explaining black box models. **Article**.

[Gunning and Aha, 2019] Gunning, D. and Aha, D. (2019). Darpa's explainable ai (xai) program. AI Magazine, 40(2):44–58. **Article**.

[Guo et al.,] Guo, X., Gao, L., Liu, X., and Yin, J. Improved deep embedded clustering with local structure preservation. In Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, pages 1753–1759. International Joint Conferences on Artificial Intelligence Organization.

[Haas et al., 2024] Haas, R., Huberman-Spiegelglas, I., Mulayoff, R., Graßhof, S., Brandt, S. S., and Michaeli, T. (2024). Discovering interpretable directions in the semantic latent space of diffusion models. In 2024 IEEE 18th International Conference on Automatic Face and Gesture Recognition (FG), pages 1–9. IEEE.

[He et al., 2017] He, X., Liao, L., Zhang, H., Nie, L., Hu, X., and Chua, T.-S. (2017). Neural collaborative filtering.

[Hessel et al., 2017] Hessel, M., Modayil, J., van Hasselt, H., Schaul, T., Ostrovski, G., Dabney, W., Horgan, D., Piot, B., Azar, M., and Silver, D. (2017). Rainbow: Combining improvements in deep reinforcement learning.

[Hilbert and López, 2011] Hilbert, M. and López, P. (2011). The world's technological capacity to store, communicate, and compute information. science, 332(6025):60–65.

[Hochreiter and Schmidhuber, 1997] Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. Neural computation, 9(8):1735–1780.

[Hornik et al., 1989] Hornik, K., Stinchcombe, M., and White, H. (1989). Multilayer feedforward networks are universal approximators. Neural networks, 2(5):359–366.

[Howard, 1960] Howard, R. A. (1960). Dynamic programming and markov processes. MIT Press.

[Huber et al., 2019] Huber, T., Schiller, D., and André, E. (2019). Enhancing explainability of deep reinforcement learning through selective layer-wise relevance propagation. In KI 2019: Advances in Artificial Intelligence: 42nd German Conference on AI, Kassel, Germany, September 23–26, 2019, Proceedings 42, pages 188–202. Springer. **Article**.

[Hubert and Arabie, 1985] Hubert, L. and Arabie, P. (1985). Comparing partitions. Journal of classification, 2(1):193–218. **Article**.

[Itaya et al., 2021] Itaya, H., Hirakawa, T., Yamashita, T., Fujiyoshi, H., and Sugiura, K. (2021). Visual explanation using attention mechanism in actor-critic-based deep reinforcement learning. **Article**.

[Jermyn et al., 2024] Jermyn, A., Templeton, A., Batson, J., and Bricken, T. (2024). Tanh penalty in dictionary learning. **Web**.

[Jumper et al., 2021] Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., Žídek, A., Potapenko, A., et al. (2021). Highly accurate protein structure prediction with alphafold. nature, 596(7873):583–589.

[Juozapaitis et al., 2019] Juozapaitis, Z., Koul, A., Fern, A., Erwig, M., and Doshi-Velez, F. (2019). Explainable reinforcement learning via reward decomposition. In IJCAI/ECAI Workshop on explainable artificial intelligence. **Article**.

[Karvonen, 2024] Karvonen, A. (2024). Emergent world models and latent variable estimation in chess-playing language models.

[Kawaguchi et al., 2022] Kawaguchi, K., Bengio, Y., and Kaelbling, L. (2022). Generalization in Deep Learning, page 112–148. Cambridge University Press.

[Knuth and Moore, 1975] Knuth, D. E. and Moore, R. W. (1975). An analysis of alpha-beta pruning. Artificial intelligence, 6(4):293–326. **Article**.

[Krizhevsky et al., 2012] Krizhevsky, A., Sutskever, I., and Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. Advances in neural information processing systems, 25.

[LeCun et al., 1989] LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., and Jackel, L. D. (1989). Backpropagation applied to handwritten zip code recognition. Neural computation, 1(4):541–551.

[Lewis et al., 2021] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., tau Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. (2021). Retrieval-augmented generation for knowledge-intensive nlp tasks.

[Li et al., 2024] Li, K., Hopkins, A. K., Bau, D., Viégas, F., Pfister, H., and Wattenberg, M. (2024). Emergent world representations: Exploring a sequence model trained on a synthetic task.

[Lillicrap et al., 2015] Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., and Wierstra, D. (2015). Continuous control with deep reinforcement learning.

[Lu et al., 2019] Lu, G., Ouyang, W., Xu, D., Zhang, X., Cai, C., and Gao, Z. (2019). Dvc: An end-to-end deep video compression framework.

[MacQueen, 1967] MacQueen, J. (1967). Multivariate observations. In Proceedings ofthe 5th Berkeley Symposium on Mathematical Statisticsand Probability, volume 1, pages 281–297. **Article**.

[Madumal et al., 2020] Madumal, P., Miller, T., Sonenberg, L., and Vetere, F. (2020). Explainable reinforcement learning through a causal lens. In Proceedings of the AAAI conference on artificial intelligence, volume 34, pages 2493–2500. **Article**.

[Marconato et al., 2023] Marconato, E., Passerini, A., and Teso, S. (2023). Interpretability is in the mind of the beholder: A causal framework for human-interpretable representation learning.

[McCulloch and Pitts, 1943] McCulloch, W. S. and Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. The bulletin of mathematical biophysics, 5(4):115–133.

[Miklautz et al.,] Miklautz, L., Klein, T., Sidak, K., Leiber, C., Lang, T., Shkabrii, A., Tschiatschek, S., and Plant, C. Breaking the reclustering barrier in centroid-based deep clustering.

[Miller, 2019] Miller, T. (2019). Explanation in artificial intelligence: Insights from the social sciences. Artificial Intelligence, 267:1–38. **Article**.

[Minsky and Papert, 1969] Minsky, M. and Papert, S. (1969). An introduction to computational geometry. Cambridge tiass., HIT, 479(480):104.

[Misak, 2013] Misak, C. (2013). The American Pragmatists. Oxford University Press.

[Mitchell, 1997] Mitchell, T. M. (1997). Machine Learning. McGraw-Hill, New York.

[Mnih et al., 2013] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. (2013). Playing atari with deep reinforcement learning.

[Mott et al., 2019] Mott, A., Zoran, D., Chrzanowski, M., Wierstra, D., and Jimenez Rezende, D. (2019). Towards interpretable reinforcement learning using attention augmented agents. Advances in neural information processing systems, 32. **Article**.

[Newell, 1980] Newell, A. (1980). Physical symbol systems. Cognitive Science, 4(2):135–183.

[Olah et al., 2020] Olah, C., Cammarata, N., Schubert, L., Goh, G., Petrov, M., and Carter, S. (2020). Zoom in: An introduction to circuits. Distill. **Article**.

[Olson et al., 2021] Olson, M. L., Khanna, R., Neal, L., Li, F., and Wong, W.-K. (2021). Counterfactual state explanations for reinforcement learning agents via generative deep learning. Artificial Intelligence, 295:103455. **Article**. **Code**.

[OpenAI et al., 2019] OpenAI, :, Berner, C., Brockman, G., Chan, B., Cheung, V., Dębiak, P., Dennison, C., Farhi, D., Fischer, Q., Hashme, S., Hesse, C., Józefowicz, R., Gray, S., Olsson, C., Pachocki, J., Petrov, M., d. O. Pinto, H. P., Raiman, J., Salimans, T., Schlatter, J., Schneider, J., Sidor, S., Sutskever, I., Tang, J., Wolski, F., and Zhang, S. (2019). Dota 2 with large scale deep reinforcement learning.

[Ouyang et al., 2022] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., and Lowe, R. (2022). Training language models to follow instructions with human feedback.

[Payani and Fekri, 2019] Payani, A. and Fekri, F. (2019). Inductive logic programming via differentiable deep neural logic networks. **Article**. **Code**.

[Peirce, 1878] Peirce, C. S. (1878). How to make our ideas clear. Popular Science Monthly, 12:286–302.

[Peyré and Cuturi, 2020] Peyré, G. and Cuturi, M. (2020). Computational optimal transport. **Article**.

[Radford et al., 2021] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I. (2021). Learning transferable visual models from natural language supervision.

[Raina et al., 2009] Raina, R., Madhavan, A., Ng, A. Y., et al. (2009). Large-scale deep unsupervised learning using graphics processors. In Icml, volume 9, pages 873–880.

[Rand, 1971] Rand, W. M. (1971). Objective criteria for the evaluation of clustering methods. Journal of the American Statistical association, 66(336):846–850.

[Ren et al.,] Ren, Y., Pu, J., Yang, Z., Xu, J., Li, G., Pu, X., Yu, P. S., and He, L. Deep clustering: A comprehensive survey.

[Rosenblatt, 1958] Rosenblatt, F. (1958). The perceptron: a probabilistic model for information storage and organization in the brain. Psychological review, 65(6):386.

[Ross and Bagnell, 2010] Ross, S. and Bagnell, D. (2010). Efficient reductions for imitation learning. In Teh, Y. W. and Titterington, M., editors, Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, volume 9 of Proceedings of Machine Learning Research, pages 661–668, Chia Laguna Resort, Sardinia, Italy. PMLR. **Article**.

[Ross et al., 2011] Ross, S., Gordon, G., and Bagnell, D. (2011). A reduction of imitation learning and structured prediction to no-regret online learning. In Gordon, G., Dunson, D., and Dudík, M., editors, Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, volume 15 of Proceedings of Machine Learning Research, pages 627–635, Fort Lauderdale, FL, USA. PMLR. **Article**.

[Rudin, 2019] Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. Nature Machine Intelligence, 1(5):206–215. **Article**.

[Rumelhart et al., 1986] Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. nature, 323(6088):533–536.

[Samvelyan et al., 2019] Samvelyan, M., Rashid, T., de Witt, C. S., Farquhar, G., Nardelli, N., Rudner, T. G. J., Hung, C.-M., Torr, P. H. S., Foerster, J., and Whiteson, S. (2019). The starcraft multi-agent challenge.

[Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms.

[Sequeira and Gervasio, 2020] Sequeira, P. and Gervasio, M. (2020). Interestingness elements for explainable reinforcement learning: Understanding agents' capabilities and limitations. Artificial Intelligence, 288:103367. **Article**.

[Seshia et al., 2022] Seshia, S. A., Sadigh, D., and Sastry, S. S. (2022). Toward verified artificial intelligence. Communications of the ACM, 65(7):46–55. **Article**.

[Shao et al., 2024] Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., and Guo, D. (2024). Deepseekmath: Pushing the limits of mathematical reasoning in open language models.

[Shlens, 2014] Shlens, J. (2014). Notes on kullback-leibler divergence and likelihood. **Article**.

[Siegel and Craver, 2024] Siegel, G. and Craver, C. F. (2024). Phenomenological laws and mechanistic explanations. Philosophy of Science, 91(1):132–150.

[Sieusahai and Guzdial, 2021] Sieusahai, A. and Guzdial, M. (2021). Explaining deep reinforcement learning agents in the atari domain through a surrogate model. **Article**.

[Silver et al., 2017a] Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., et al. (2017a). Mastering chess and shogi by self-play with a general reinforcement learning algorithm. arXiv preprint arXiv:1712.01815.

[Silver et al., 2017b] Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., and Hassabis, D. (2017b). Mastering chess and shogi by self-play with a general reinforcement learning algorithm.

[Silver et al., 2021] Silver, D., Singh, S., Precup, D., and Sutton, R. S. (2021). Reward is enough. Artificial intelligence, 299:103535.

[Simon, 1996] Simon, H. A. (1996). The Sciences of the Artificial. MIT Press, 3 edition.

[Simonyan et al., 2014] Simonyan, K., Vedaldi, A., and Zisserman, A. (2014). Deep inside convolutional networks: Visualising image classification models and saliency maps. **Article**. **Code**.

[Skinner, 1938] Skinner, B. F. (1938). The behavior of organisms: An experimental analysis. Appleton-Century.

[Sutton, 1988] Sutton, R. S. (1988). Learning to predict by the methods of temporal differences. Machine Learning, 3:9–44.

[Sutton and Barto, 1998] Sutton, R. S. and Barto, A. G. (1998). Reinforcement Learning: An Introduction. MIT Press, 1st edition.

[Sutton and Barto, 2018] Sutton, R. S. and Barto, A. G. (2018). Reinforcement Learning: An Introduction. MIT Press. **Book**.

[Telgarsky, 2016] Telgarsky, M. (2016). Benefits of depth in neural networks. In Conference on learning theory, pages 1517–1539. PMLR.

[Templeton et al., 2024] Templeton, A., Conerly, T., Marcus, J., Lindsey, J., Bricken, T., Chen, B., et al. (2024). Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet. Transformer Circuits Thread. **Web**.

[Thorndike, 1911] Thorndike, E. L. (1911). Animal intelligence: Experimental studies. The Psychological Review Monograph Supplement, 8(4).

[Topin et al., 2021] Topin, N., Milani, S., Fang, F., and Veloso, M. (2021). Iterative bounding mdps: Learning interpretable policies via non-interpretable methods. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 35, pages 9923–9931. **Article**.

[Trivedi et al., 2022] Trivedi, D., Zhang, J., Sun, S.-H., and Lim, J. J. (2022). Learning to synthesize programs as interpretable and generalizable policies. **Article**. **Code**.

[Tsitsiklis and Van Roy, 1996] Tsitsiklis, J. and Van Roy, B. (1996). Analysis of temporal-diffference learning with function approximation. Advances in neural information processing systems, 9.

[Turing, 1948] Turing, A. (1948). Intelligent machinery. National Physical Laboratory Report.

[Vaswani et al., 2017] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. Advances in neural information processing systems, 30.

[Vaswani et al., 2023] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2023). Attention is all you need. **Article**.

[Voynov and Babenko, 2020] Voynov, A. and Babenko, A. (2020). Unsupervised discovery of interpretable directions in the gan latent space.

[Watkins, 1989] Watkins, C. J. C. H. (1989). Learning from delayed rewards. PhD Thesis, University of Cambridge.

[Watkins and Dayan, 1992] Watkins, C. J. C. H. and Dayan, P. (1992). Q-learning. Machine Learning, 8:279–292.

[Wei et al.,] Wei, X., Zhang, Z., Huang, H., and Zhou, Y. An overview on deep clustering. 590:127761.

[Weitkamp et al., 2019] Weitkamp, L., van der Pol, E., and Akata, Z. (2019). Visual rationalizations in deep reinforcement learning for atari games. **Article**.

[Xie et al.,] Xie, J., Girshick, R., and Farhadi, A. Unsupervised deep embedding for clustering analysis.

[Zahavy et al., 2017] Zahavy, T., Zrihem, N. B., and Mannor, S. (2017). Graying the black box: Understanding dqns. **Article**.

[Zednik, 2019] Zednik, C. (2019). Solving the black box problem: A normative framework for explainable artificial intelligence. arXiv preprint arXiv:1903.04361.